

```
# Google drive set up
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
!pip install -q torchmetrics
!pip install -U albumentations
```

 [Show hidden output](#)

✓ Imports and Data Processing

```
import os
import json
import shutil
import zipfile
import numpy as np
from PIL import Image
import tifffile as tiff
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision.transforms import v2
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
import albumentations as A
from albumentations.pytorch import ToTensorV2
from torchmetrics.detection.mean_ap import MeanAveragePrecision
from tqdm import tqdm

# Extract Tier_1 (Training)
tier1_base = '/content/drive/MyDrive/Building Damage Assessment/Tier_1'
tier1_images_zip = os.path.join(tier1_base, 'images.zip')
tier1_labels_zip = os.path.join(tier1_base, 'labels.zip')

if os.path.exists(tier1_images_zip):
    with zipfile.ZipFile(tier1_images_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/tier1')
    print('Tier1 images extracted')

if os.path.exists(tier1_labels_zip):
    with zipfile.ZipFile(tier1_labels_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/tier1_labels')
    print('Tier1 labels extracted')

# Extract Holdout (Validation)
holdout_base = '/content/drive/MyDrive/Building Damage Assessment/Holdout'
holdout_images_zip = os.path.join(holdout_base, 'images.zip')
holdout_labels_zip = os.path.join(holdout_base, 'labels.zip')

if os.path.exists(holdout_images_zip):
```

```

with zipfile.ZipFile(holdout_images_zip, 'r') as zip_ref:
    zip_ref.extractall('/content/holdout')
print('Holdout images extracted')

if os.path.exists(holdout_labels_zip):
    with zipfile.ZipFile(holdout_labels_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/holdout_labels')
    print('Holdout labels extracted')

```

↗ Tier1 images extracted
 Tier1 labels extracted
 Holdout images extracted
 Holdout labels extracted

✓ Dataset creation and Data Loader

```

class BuildingDataset(Dataset):
    def __init__(self, image_dir, annotation_path, transforms=None):
        self.image_dir = image_dir
        self.transforms = transforms

        # Load custom annotation format
        with open(annotation_path, 'r') as f:
            self.annotations = json.load(f)

        self.image_names = list(self.annotations.keys())
        print(f"Loaded {len(self.image_names)} images")

    def __len__(self):
        return len(self.image_names)

    def __getitem__(self, idx):
        filename = self.image_names[idx]
        image_path = os.path.join(self.image_dir, filename)

        # --- Load image (.tif support) ---
        try:
            image_array = tifff.imread(image_path)
        except Exception as e:
            raise RuntimeError(f"Failed to read image: {image_path} - {e}")

        # Convert grayscale or CHW to HWC RGB
        if image_array.ndim == 2:
            image_array = np.stack([image_array] * 3, axis=-1)
        elif image_array.ndim == 3:
            if image_array.shape[0] in [1, 3] and image_array.shape[0] < image_array.shape[-1]:
                image_array = np.transpose(image_array, (1, 2, 0))
            if image_array.shape[2] == 1:
                image_array = np.repeat(image_array, 3, axis=2)

        # Get image dimensions for bbox conversion
        img_height, img_width = image_array.shape[:2]

        # Convert to uint8 if needed
        if image_array.dtype != np.uint8:

```

```

image_array = image_array.astype(np.float32)
if image_array.max() > 1.0:
    # Normalize to 0-255 range
    image_array = 255 * (image_array - image_array.min()) / (np.ptp(image_array) + 1e-6)
else:
    # Already in 0-1 range
    image_array = image_array * 255
image_array = image_array.astype(np.uint8)

boxes = []
labels = []

# Handle both annotation formats properly
for ann in self.annotations[filename]:
    bbox = ann['bbox']

    # Check if bbox is in [x, y, w, h] or [x1, y1, x2, y2] format
    if len(bbox) == 4:
        x1, y1, x2_or_w, y2_or_h = bbox

        # Detect format based on values
        if x2_or_w > img_width or y2_or_h > img_height:
            # Likely [x1, y1, x2, y2] format
            x2, y2 = x2_or_w, y2_or_h
        elif x2_or_w < x1 or y2_or_h < y1:
            # Likely [x, y, w, h] format where w, h are small
            x2 = x1 + x2_or_w
            y2 = y1 + y2_or_h
        else:
            # Ambiguous - assume [x, y, w, h] if w,h look like dimensions
            if x2_or_w < img_width/2 and y2_or_h < img_height/2:
                # Likely width/height
                x2 = x1 + x2_or_w
                y2 = y1 + y2_or_h
            else:
                # Likely coordinates
                x2, y2 = x2_or_w, y2_or_h

        # Ensure valid bounds
        x1 = max(0, min(img_width-1, x1))
        y1 = max(0, min(img_height-1, y1))
        x2 = max(x1+1, min(img_width, x2))
        y2 = max(y1+1, min(img_height, y2))

        # Only add valid boxes
        if x2 > x1 and y2 > y1:
            boxes.append([x1, y1, x2, y2])
            labels.append(ann.get('label', 1)) # default label is 1 (building)

# Handle empty boxes case
if len(boxes) == 0:
    print(f"Warning: No valid boxes found for {filename}")
    boxes = [[0, 0, 10, 10]] # Dummy box
    labels = [1]

# Convert to proper format for albugmentations

```

```

boxes = np.array(boxes, dtype=np.float32)
labels = np.array(labels, dtype=np.int64)

# Apply transforms
if self.transforms:
    try:
        transformed = self.transforms(
            image=image_array,
            bboxes=boxes.tolist(),
            labels=labels.tolist()
        )
        image = transformed['image']
        boxes = torch.tensor(transformed['bboxes'], dtype=torch.float32)
        labels = torch.tensor([int(label) for label in transformed['labels']], dtype=torch.int64)

        # Validate transformed boxes
        if len(boxes) == 0:
            boxes = torch.tensor([[0, 0, 10, 10]], dtype=torch.float32)
            labels = torch.tensor([1], dtype=torch.int64)

    except Exception as e:
        print(f"Transform error for {filename}: {e}")
        # Fallback without transforms
        image = torchvision.transforms.ToTensor()(Image.fromarray(image_array))
        boxes = torch.tensor(boxes, dtype=torch.float32)
        labels = torch.tensor([int(label) for label in transformed['labels']], dtype=torch.int64)
else:
    image = torchvision.transforms.ToTensor()(Image.fromarray(image_array))
    boxes = torch.tensor(boxes, dtype=torch.float32)
    labels = torch.tensor([int(label) for label in transformed['labels']], dtype=torch.int64)

target = {
    'boxes': boxes,
    'labels': labels,
    'image_id': torch.tensor([idx])
}

return image, target

def get_transforms(train=True):
    if train:
        return A.Compose([
            A.HorizontalFlip(p=0.3),
            A.RandomBrightnessContrast(brightness_limit=0.1, contrast_limit=0.1, p=0.2),
            A.Resize(512, 512), # Smaller size for memory efficiency
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2()
        ], bbox_params=A.BboxParams(
            format='pascal_voc',
            label_fields=['labels'],
            min_visibility=0.3,
            min_area=25.0,
            check_each_transform=True
        ))
    else:
        return A.Compose([

```

```

        A.Resize(512, 512),
        A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ToTensorV2())
    ], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

def collate_fn(batch):
    """Custom collate function"""
    images, targets = tuple(zip(*batch))
    return list(images), list(targets)

class BuildingDataset(Dataset):
    def __init__(self, image_dir, annotation_path, transforms=None):
        self.image_dir = image_dir
        self.transforms = transforms

        # Load custom annotation format
        with open(annotation_path, 'r') as f:
            self.annotations = json.load(f)

        self.image_names = list(self.annotations.keys())
        print(f"Loaded {len(self.image_names)} images")

    def __len__(self):
        return len(self.image_names)

    def __getitem__(self, idx):
        filename = self.image_names[idx]
        image_path = os.path.join(self.image_dir, filename)

        try:
            image_array = tiff.imread(image_path)
        except Exception as e:
            raise RuntimeError(f"Failed to read image: {image_path} - {e}")

        # Convert grayscale or CHW to HWC RGB
        if image_array.ndim == 2:
            image_array = np.stack([image_array] * 3, axis=-1)
        elif image_array.ndim == 3:
            if image_array.shape[0] in [1, 3] and image_array.shape[0] < image_array.shape[-1]:
                image_array = np.transpose(image_array, (1, 2, 0))
            if image_array.shape[2] == 1:
                image_array = np.repeat(image_array, 3, axis=2)

        # Get image dimensions for bbox conversion
        img_height, img_width = image_array.shape[:2]

        # Convert to uint8 if needed
        if image_array.dtype != np.uint8:
            image_array = image_array.astype(np.float32)
            if image_array.max() > 1.0:
                # Normalize to 0-255 range
                image_array = 255 * (image_array - image_array.min()) / (np.ptp(image_array) + 1e-6)
            else:
                # Already in 0-1 range
                image_array = image_array * 255
            image_array = image_array.astype(np.uint8)

```

```

boxes = []
labels = []

# **CRITICAL FIX**: Handle both annotation formats properly
for ann in self.annotations[filename]:
    bbox = ann['bbox']

    # Check if bbox is in [x, y, w, h] or [x1, y1, x2, y2] format
    if len(bbox) == 4:
        x1, y1, x2_or_w, y2_or_h = bbox

        # Detect format based on values
        if x2_or_w > img_width or y2_or_h > img_height:
            # Likely [x1, y1, x2, y2] format
            x2, y2 = x2_or_w, y2_or_h
        elif x2_or_w < x1 or y2_or_h < y1:
            # Likely [x, y, w, h] format where w, h are small
            x2 = x1 + x2_or_w
            y2 = y1 + y2_or_h
        else:
            # Ambiguous - assume [x, y, w, h] if w,h look like dimensions
            if x2_or_w < img_width/2 and y2_or_h < img_height/2:
                # Likely width/height
                x2 = x1 + x2_or_w
                y2 = y1 + y2_or_h
            else:
                # Likely coordinates
                x2, y2 = x2_or_w, y2_or_h

        # Ensure valid bounds
        x1 = max(0, min(img_width-1, x1))
        y1 = max(0, min(img_height-1, y1))
        x2 = max(x1+1, min(img_width, x2))
        y2 = max(y1+1, min(img_height, y2))

        # Only add valid boxes
        if x2 > x1 and y2 > y1:
            boxes.append([x1, y1, x2, y2])
            labels.append(int(ann.get('label', 1))) # ✅ FIXED: Ensure integer from start

# Handle empty boxes case
if len(boxes) == 0:
    print(f"Warning: No valid boxes found for {filename}")
    boxes = [[0, 0, 10, 10]] # Dummy box
    labels = [1]

# Convert to proper format for albuementations
boxes = np.array(boxes, dtype=np.float32)
labels = np.array(labels, dtype=np.int32) # ✅ FIXED: Use int32 to ensure integer type

# Apply transforms
if self.transforms:
    try:
        transformed = self.transforms(
            image=image_array,

```

```

        bboxes=boxes.tolist(),
        labels=labels.tolist()
    )
    image = transformed['image']
    boxes = torch.tensor(transformed['bboxes'], dtype=torch.float32)

    labels = torch.tensor([int(label) for label in transformed['labels']], dtype=torch.int64)

    # Validate transformed boxes
    if len(boxes) == 0:
        boxes = torch.tensor([[0, 0, 10, 10]], dtype=torch.float32)
        labels = torch.tensor([1], dtype=torch.int64)

    except Exception as e:
        print(f"Transform error for {filename}: {e}")
        # Fallback without transforms
        image = torchvision.transforms.ToTensor()(Image.fromarray(image_array))
        boxes = torch.tensor(boxes, dtype=torch.float32)
        labels = torch.tensor(labels, dtype=torch.int64)
    else:
        image = torchvision.transforms.ToTensor()(Image.fromarray(image_array))
        boxes = torch.tensor(boxes, dtype=torch.float32)
        labels = torch.tensor(labels, dtype=torch.int64)

    target = {
        'boxes': boxes,
        'labels': labels,
        'image_id': torch.tensor([idx])
    }

    return image, target

def get_transforms(train=True):
    """Conservative transforms to avoid bbox issues"""
    if train:
        return A.Compose([
            A.HorizontalFlip(p=0.3),
            A.RandomBrightnessContrast(brightness_limit=0.1, contrast_limit=0.1, p=0.2),
            A.Resize(512, 512), # Smaller size for memory efficiency
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2()
        ], bbox_params=A.BboxParams(
            format='pascal_voc',
            label_fields=['labels'],
            min_visibility=0.3,
            min_area=25.0,
            check_each_transform=True
        ))
    else:
        return A.Compose([
            A.Resize(512, 512),
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2()
        ], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

def collate_fn(batch):

```

```
"""Custom collate function"""
images, targets = tuple(zip(*batch))
return list(images), list(targets)
```

▼ Training and Validation Functions

```
def train_epoch(model, optimizer, data_loader, device, epoch):
    model.train()
    total_loss = 0
    num_batches = 0

    progress_bar = tqdm(data_loader, desc=f"Training Epoch {epoch}")

    for batch_idx, (images, targets) in enumerate(progress_bar):
        try:
            images = [img.to(device) for img in images]
            targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

            # Skip batches with invalid boxes
            valid_batch = True
            for target in targets:
                if len(target['boxes']) == 0:
                    valid_batch = False
                    break
            # Check for valid box coordinates
            boxes = target['boxes']
            if torch.any(boxes[:, 2] <= boxes[:, 0]) or torch.any(boxes[:, 3] <= boxes[:, 1]):
                valid_batch = False
                break

            if not valid_batch:
                continue

            # Forward pass
            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())

            # Check for invalid loss
            if not torch.isfinite(losses):
                print(f"Warning: Invalid loss detected at batch {batch_idx}, skipping")
                continue

            # Backward pass
            optimizer.zero_grad()
            losses.backward()

            # Gradient clipping to prevent exploding gradients
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

            optimizer.step()

            total_loss += losses.item()
            num_batches += 1
            progress_bar.set_postfix(loss=losses.item())
```



```

    except Exception as e:
        print(f"Error in training batch {batch_idx}: {e}")
        continue

    avg_loss = total_loss / max(num_batches, 1)
    print(f" Training Loss: {avg_loss:.4f}")
    return avg_loss

def validate_epoch(model, data_loader, device, epoch):
    model.eval()
    # Use both mAP@0.5 and mAP@0.75 for better evaluation
    metric = MeanAveragePrecision(iou_thresholds=[0.5, 0.75])

    total_val_loss = 0.0
    num_batches = 0
    total_predictions = 0
    high_conf_predictions = 0

    progress_bar = tqdm(data_loader, desc=f"Validating Epoch {epoch}")

    with torch.no_grad():
        for batch_idx, (images, targets) in enumerate(progress_bar):
            try:
                images = [img.to(device) for img in images]
                targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

                # Calculate validation loss
                model.train()
                loss_dict = model(images, targets)
                losses = sum(loss for loss in loss_dict.values())

                if torch.isfinite(losses):
                    total_val_loss += losses.item()
                    num_batches += 1

                # Get predictions
                model.eval()
                outputs = model(images)

                # Prepare for metric calculation
                preds = []
                targets_clean = []

                for output, target in zip(outputs, targets):
                    # Count total predictions
                    total_predictions += len(output['scores'])
                    high_conf_predictions += (output['scores'] > 0.5).sum().item()

                    # Use 0.1 confidence threshold for mAP calculation (standard)
                    # but track higher confidence predictions separately
                    valid_indices = output['scores'] > 0.1
                    if valid_indices.sum() > 0:
                        preds.append({
                            'boxes': output['boxes'][valid_indices].cpu(),
                            'scores': output['scores'][valid_indices].cpu(),

```

```

        'labels': output['labels'][valid_indices].cpu()
    })
else:
    # Add very low confidence dummy if no predictions
    preds.append({
        'boxes': torch.tensor([[0, 0, 1, 1]], dtype=torch.float32),
        'scores': torch.tensor([0.001], dtype=torch.float32),
        'labels': torch.tensor([1], dtype=torch.int64)
    })

    targets_clean.append({
        'boxes': target['boxes'].cpu(),
        'labels': target['labels'].cpu()
    })

    metric.update(preds, targets_clean)
    progress_bar.set_postfix(loss=losses.item() if torch.isfinite(losses) else 0.0)

except Exception as e:
    print(f"Error in validation batch {batch_idx}: {e}")
    continue

try:
    result = metric.compute()
    val_loss = total_val_loss / max(num_batches, 1)
    mAP_50 = result['map_50'].item()
    mAP_75 = result['map_75'].item()

    # Calculate prediction statistics
    avg_predictions = total_predictions / max(len(data_loader) * 2, 1) # batch_size=2
    high_conf_ratio = high_conf_predictions / max(total_predictions, 1)

except Exception as e:
    print(f"Error computing metrics: {e}")
    val_loss = total_val_loss / max(num_batches, 1)
    mAP_50 = 0.0
    mAP_75 = 0.0
    avg_predictions = 0.0
    high_conf_ratio = 0.0

print(f" Validation Loss: {val_loss:.4f}")
print(f" mAP@0.5: {mAP_50:.4f} | mAP@0.75: {mAP_75:.4f}")
print(f" Avg predictions/image: {avg_predictions:.1f} | High confidence (>0.5): {high_conf_ratio:.1%}")

return val_loss, mAP_50, mAP_75

def main_training(model, train_loader, val_loader, optimizer, scheduler, device,
                  num_epochs=15, patience=5, model_save_path='best_model_fixed.pth'):

    best_map = 0.0
    best_epoch = 0
    epochs_without_improvement = 0

    history = {
        'train_loss': [],
        'val_loss': [],

```

```

    'mAP_50': [],
    'mAP_75': []
}

print(f"\n Training (No Early Stopping)")

for epoch in range(1, num_epochs + 1):
    try:
        print(f"\n{' '*50}")
        print(f"EPOCH {epoch}/{num_epochs}")
        print(f"{' '*50}")

        train_loss = train_epoch(model, optimizer, train_loader, device, epoch)
        val_loss, mAP_50, mAP_75 = validate_epoch(model, val_loader, device, epoch)

        if scheduler is not None:
            scheduler.step()
            current_lr = scheduler.get_last_lr()[0]
            print(f" Learning rate: {current_lr:.6f}")

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['mAP_50'].append(mAP_50)
        history['mAP_75'].append(mAP_75)

        # Use mAP@0.5 as the main metric for saving best model
        if mAP_50 > best_map:
            best_map = mAP_50
            best_epoch = epoch
            epochs_without_improvement = 0

            torch.save(model.state_dict(), model_save_path)
            print(f" Model improved! Saved at epoch {epoch}")
            print(f" Best mAP@0.5: {mAP_50:.4f} | Best mAP@0.75: {mAP_75:.4f}")
        else:
            epochs_without_improvement += 1
            print(f" No improvement for {epochs_without_improvement} epochs")
            print(f" Current: mAP@0.5={mAP_50:.4f} | Best: {best_map:.4f} (epoch {best_epoch})")

        if epochs_without_improvement >= patience:
            print(f"\n Early stopping triggered at epoch {epoch}")
            print(f" Final best mAP@0.5: {best_map:.4f} (epoch {best_epoch})")
            break

    except Exception as e:
        print(f"Error in epoch {epoch}: {e}")
        continue

return history

# Create datasets (when using Google Colab)
train_dataset_fixed = BuildingDataset(
    '/content/tier1/images',
    '/content/drive/MyDrive/Building Damage Assessment/Annotations/tier1_annotations.json',
    transforms=get_transforms(train=True)
)

```

```
val_dataset_fixed = BuildingDataset(
    '/content/holdout/images',
    '/content/drive/MyDrive/Building Damage Assessment/Annotations/holdout_annotations.json',
    transforms=get_transforms(train=False)
)
```

```
print(f"Train dataset size: {len(train_dataset_fixed)}")
print(f"Validation dataset size: {len(val_dataset_fixed)}")
```

```
↳ Loaded 4566 images
   Loaded 1526 images
   Train dataset size: 4566
   Validation dataset size: 1526
```

```
# Create datasets (when running locally)
```

```
train_dataset_fixed = BuildingDataset(
    r'C:\Users\sajja\Desktop\Dissertation\geotiffs\tier1\images',
    r'C:\Users\sajja\Desktop\Dissertation\Annotations\tier1_annotations.json',
    transforms=get_transforms(train=True)
)
```

```
val_dataset_fixed = BuildingDataset(
    r'C:\Users\sajja\Desktop\Dissertation\geotiffs\hold\images',
    r'C:\Users\sajja\Desktop\Dissertation\Annotations\holdout_annotations.json',
    transforms=get_transforms(train=False)
)
```

```
print(f"Train dataset size: {len(train_dataset_fixed)}")
print(f"Validation dataset size: {len(val_dataset_fixed)}")
```

```
↳ Loaded 4566 images
   Loaded 1526 images
   Train dataset size: 4566
   Validation dataset size: 1526
```

```
# Create data loaders with smaller batch size
```

```
train_loader = DataLoader(
    train_dataset_fixed,
    batch_size=4,
    shuffle=True,
    collate_fn=collate_fn,
    num_workers=2
)
```

```
val_loader = DataLoader(
    val_dataset_fixed,
    batch_size=4,
    shuffle=False,
    collate_fn=collate_fn,
    num_workers=2
)
```

```
import torchvision.models.detection as detection
```

```
# Get all detection model functions (exclude weights classes and private methods)
models = [name for name in dir(detection)
          if not name.startswith('_')
          and not name.endswith('_Weights')
          and callable(getattr(detection, name))
          and name not in ['_utils']]
```

```
print("PyTorch Pre-trained Object Detection Models:")
print("=" * 50)
for model in sorted(models):
    print(f" • {model}")
```



PyTorch Pre-trained Object Detection Models:

=====

- FCOS
- FasterRCNN
- KeypointRCNN
- MaskRCNN
- RetinaNet
- fasterrcnn_mobilenet_v3_large_320_fpn
- fasterrcnn_mobilenet_v3_large_fpn
- fasterrcnn_resnet50_fpn
- fasterrcnn_resnet50_fpn_v2
- fcos_resnet50_fpn
- keypointrcnn_resnet50_fpn
- maskrcnn_resnet50_fpn
- maskrcnn_resnet50_fpn_v2
- retinanet_resnet50_fpn
- retinanet_resnet50_fpn_v2
- ssd300_vgg16
- ssdlite320_mobilenet_v3_large

✓ ResNet50

```
from torchvision.models.detection import fasterrcnn_resnet50_fpn, FasterRCNN_ResNet50_FPN_Weights
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
```

```
def get_model(num_classes=2):
    model = fasterrcnn_resnet50_fpn(weights=FasterRCNN_ResNet50_FPN_Weights.DEFAULT)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = get_model(num_classes=2)
model.to(device)
```

```
optimizer = torch.optim.SGD(
    [p for p in model.parameters() if p.requires_grad],
    lr=0.002, # Lower learning rate
    momentum=0.9,
    weight_decay=0.0005
)
```

```
# Learning rate scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path = os.path.join(model_dir, 'Model.pth')

history = main_training(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    scheduler=lr_scheduler,
    device=device,
    num_epochs=15,
    patience=5,
    model_save_path=model_save_path
)
```



Training (patience=5)

```
=====
EPOCH 1/15
=====
Training Epoch 1: 100%|██████████| 2283/2283 [19:40<00:00, 1.93it/s, loss=1.61]
Training Loss: 1.4092
Validating Epoch 1: 100%|██████████| 763/763 [05:59<00:00, 2.12it/s, loss=1.75]
Validation Loss: 1.4114
mAP@0.5: 0.2065 | mAP@0.75: 0.0545
Avg predictions/image: 93.6 | High confidence (>0.5): 32.2%
Learning rate: 0.002000
Model improved! Saved at epoch 1
Best mAP@0.5: 0.2065 | Best mAP@0.75: 0.0545

=====
EPOCH 2/15
=====
Training Epoch 2: 100%|██████████| 2283/2283 [19:35<00:00, 1.94it/s, loss=0.9]
Training Loss: 1.2857
Validating Epoch 2: 100%|██████████| 763/763 [05:55<00:00, 2.14it/s, loss=1.73]
Validation Loss: 1.3227
mAP@0.5: 0.2326 | mAP@0.75: 0.0694
Avg predictions/image: 92.5 | High confidence (>0.5): 36.9%
Learning rate: 0.002000
Model improved! Saved at epoch 2
Best mAP@0.5: 0.2326 | Best mAP@0.75: 0.0694

=====
EPOCH 3/15
=====
Training Epoch 3: 100%|██████████| 2283/2283 [19:35<00:00, 1.94it/s, loss=1.64]
Training Loss: 1.2438
Validating Epoch 3: 100%|██████████| 763/763 [05:55<00:00, 2.15it/s, loss=1.7]
Validation Loss: 1.2969
mAP@0.5: 0.2481 | mAP@0.75: 0.0701
Avg predictions/image: 88.8 | High confidence (>0.5): 38.7%
Learning rate: 0.002000
Model improved! Saved at epoch 3
Best mAP@0.5: 0.2481 | Best mAP@0.75: 0.0701
```

```

=====
EPOCH 4/15
=====
Training Epoch 4: 100%|██████████| 2283/2283 [19:44<00:00, 1.93it/s, loss=1.15]
Training Loss: 1.2186
Validating Epoch 4: 100%|██████████| 763/763 [05:58<00:00, 2.13it/s, loss=1.68]
Validation Loss: 1.3146
mAP@0.5: 0.2629 | mAP@0.75: 0.0753
Avg predictions/image: 85.9 | High confidence (>0.5): 38.6%
Learning rate: 0.002000
Model improved! Saved at epoch 4
Best mAP@0.5: 0.2629 | Best mAP@0.75: 0.0753

=====
EPOCH 5/15
=====

```

✓ ResNet50 V2

```

from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

```

```

def get_modelV2(num_classes=2):
    weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

model = get_modelV2(num_classes=2)
model.to(device)

```

```

optimizer = torch.optim.SGD(
    [p for p in model.parameters() if p.requires_grad],
    lr=0.002,
    momentum=0.9,
    weight_decay=0.0005
)

```

```

# Learning rate scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

```

```

# Create model save directory
model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path = os.path.join(model_dir, 'ModelV2(1).pth')

```

```

# Start realistic training
history = main_training(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,

```

```

    scheduler=lr_scheduler,
    device=device,
    num_epochs=50,
    patience=8,
    model_save_path=model_save_path
)

history_save_path = os.path.join(model_dir, 'RV2(1)_training_history.json')
with open(history_save_path, 'w') as f:
    json.dump(history, f, indent=2)
print(f" Training history saved to: {history_save_path}")

""" ResNet50 V2 Results
Training (patience=8)

=====
EPOCH 1/50
=====
Training Epoch 1: 100%|██████████| 2283/2283 [27:39<00:00, 1.38it/s, loss=0.868]
  Training Loss: 1.4411
Validating Epoch 1: 100%|██████████| 763/763 [08:08<00:00, 1.56it/s, loss=1.64]
  Validation Loss: 1.2823
  mAP@0.5: 0.2224 | mAP@0.75: 0.0727
  Avg predictions/image: 88.0 | High confidence (>0.5): 41.7%
  Learning rate: 0.002000
  Model improved! Saved at epoch 1
  Best mAP@0.5: 0.2224 | Best mAP@0.75: 0.0727

=====
EPOCH 2/50
=====
Training Epoch 2: 100%|██████████| 2283/2283 [26:47<00:00, 1.42it/s, loss=1.28]
  Training Loss: 1.1910
Validating Epoch 2: 100%|██████████| 763/763 [07:51<00:00, 1.62it/s, loss=1.63]
  Validation Loss: 1.2211
  mAP@0.5: 0.2680 | mAP@0.75: 0.0884
  Avg predictions/image: 85.1 | High confidence (>0.5): 47.6%
  Learning rate: 0.002000
  Model improved! Saved at epoch 2
  Best mAP@0.5: 0.2680 | Best mAP@0.75: 0.0884

=====
EPOCH 3/50
=====
Training Epoch 3: 100%|██████████| 2283/2283 [27:00<00:00, 1.41it/s, loss=0.789]
  Training Loss: 1.1423
Validating Epoch 3: 100%|██████████| 763/763 [07:52<00:00, 1.61it/s, loss=1.65]
  Validation Loss: 1.1936
  mAP@0.5: 0.2780 | mAP@0.75: 0.0993
  Avg predictions/image: 81.0 | High confidence (>0.5): 43.7%
  Learning rate: 0.002000
  Model improved! Saved at epoch 3
  Best mAP@0.5: 0.2780 | Best mAP@0.75: 0.0993

=====
EPOCH 4/50

```



```
=====
Training Epoch 4: 34%|██████| 778/2283 [09:11<19:17, 1.30it/s, loss=0.576]Error in training batch 777: CUDA out of memory. Tried to allocate 1022.00 MiB. GPU 0 has a total capacity of 14.74 GiB
Training Epoch 4: 100%|██████████| 2283/2283 [26:59<00:00, 1.41it/s, loss=1.14]
  Training Loss: 1.1067
Validating Epoch 4: 100%|██████████| 763/763 [07:54<00:00, 1.61it/s, loss=1.47]
  Validation Loss: 1.1601
  mAP@0.5: 0.2947 | mAP@0.75: 0.1128
  Avg predictions/image: 80.4 | High confidence (>0.5): 47.2%
  Learning rate: 0.002000
Model improved! Saved at epoch 4
Best mAP@0.5: 0.2947 | Best mAP@0.75: 0.1128

=====
EPOCH 5/50
=====
Training Epoch 5: 100%|██████████| 2283/2283 [28:02<00:00, 1.36it/s, loss=1.22]
  Training Loss: 1.0815
Validating Epoch 5: 100%|██████████| 763/763 [08:11<00:00, 1.55it/s, loss=1.46]
  Validation Loss: 1.1410
  mAP@0.5: 0.3024 | mAP@0.75: 0.1122
  Avg predictions/image: 79.2 | High confidence (>0.5): 49.2%
  Learning rate: 0.002000
Model improved! Saved at epoch 5
Best mAP@0.5: 0.3024 | Best mAP@0.75: 0.1122

=====
EPOCH 6/50
=====
Training Epoch 6: 100%|██████████| 2283/2283 [28:08<00:00, 1.35it/s, loss=1.46]
  Training Loss: 1.0553
Validating Epoch 6: 100%|██████████| 763/763 [08:15<00:00, 1.54it/s, loss=1.42]
  Validation Loss: 1.1423
  mAP@0.5: 0.3038 | mAP@0.75: 0.1142
  Avg predictions/image: 77.9 | High confidence (>0.5): 49.7%
  Learning rate: 0.002000
Model improved! Saved at epoch 6
Best mAP@0.5: 0.3038 | Best mAP@0.75: 0.1142

=====
EPOCH 7/50
=====
Training Epoch 7: 100%|██████████| 2283/2283 [28:12<00:00, 1.35it/s, loss=0.492]
  Training Loss: 1.0525
Validating Epoch 7: 100%|██████████| 763/763 [08:15<00:00, 1.54it/s, loss=1.52]
  Validation Loss: 1.1446
  mAP@0.5: 0.3055 | mAP@0.75: 0.1147
  Avg predictions/image: 76.7 | High confidence (>0.5): 49.0%
  Learning rate: 0.002000
Model improved! Saved at epoch 7
Best mAP@0.5: 0.3055 | Best mAP@0.75: 0.1147

=====
EPOCH 8/50
=====
Training Epoch 8: 100%|██████████| 2283/2283 [28:09<00:00, 1.35it/s, loss=0.827]
  Training Loss: 1.0503
```

```
Validating Epoch 8: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.5]
  Validation Loss: 1.1406
  mAP@0.5: 0.3051 | mAP@0.75: 0.1147
  Avg predictions/image: 76.3 | High confidence (>0.5): 49.4%
  Learning rate: 0.000200
  No improvement for 1 epochs
  Current: mAP@0.5=0.3051 | Best: 0.3055 (epoch 7)

=====
EPOCH 9/50
=====
Training Epoch 9: 100%|██████████| 2283/2283 [28:06<00:00, 1.35it/s, loss=1.55]
  Training Loss: 1.0445
Validating Epoch 9: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.51]
  Validation Loss: 1.1391
  mAP@0.5: 0.3077 | mAP@0.75: 0.1150
  Avg predictions/image: 77.2 | High confidence (>0.5): 50.0%
  Learning rate: 0.000200
  Model improved! Saved at epoch 9
  Best mAP@0.5: 0.3077 | Best mAP@0.75: 0.1150

=====
EPOCH 10/50
=====
Training Epoch 10: 100%|██████████| 2283/2283 [28:08<00:00, 1.35it/s, loss=0.604]
  Training Loss: 1.0437
Validating Epoch 10: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.56]
  Validation Loss: 1.1371
  mAP@0.5: 0.3072 | mAP@0.75: 0.1146
  Avg predictions/image: 76.8 | High confidence (>0.5): 50.1%
  Learning rate: 0.000200
  No improvement for 1 epochs
  Current: mAP@0.5=0.3072 | Best: 0.3077 (epoch 9)

=====
EPOCH 11/50
=====
Training Epoch 11: 100%|██████████| 2283/2283 [28:09<00:00, 1.35it/s, loss=1.14]
  Training Loss: 1.0418
Validating Epoch 11: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.39]
  Validation Loss: 1.1392
  mAP@0.5: 0.3075 | mAP@0.75: 0.1146
  Avg predictions/image: 77.0 | High confidence (>0.5): 50.0%
  Learning rate: 0.000200
  No improvement for 2 epochs
  Current: mAP@0.5=0.3075 | Best: 0.3077 (epoch 9)

=====
EPOCH 12/50
=====
Training Epoch 12: 100%|██████████| 2283/2283 [28:11<00:00, 1.35it/s, loss=1.5]
  Training Loss: 1.0397
Validating Epoch 12: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.47]
  Validation Loss: 1.1363
  mAP@0.5: 0.3076 | mAP@0.75: 0.1145
  Avg predictions/image: 76.9 | High confidence (>0.5): 50.2%
```

```
Learning rate: 0.000020
No improvement for 3 epochs
Current: mAP@0.5=0.3076 | Best: 0.3077 (epoch 9)
```

```
=====
EPOCH 13/50
=====
Training Epoch 13: 100%|██████████| 2283/2283 [28:05<00:00, 1.35it/s, loss=0.598]
Training Loss: 1.0396
Validating Epoch 13: 100%|██████████| 763/763 [08:16<00:00, 1.54it/s, loss=1.55]
Validation Loss: 1.1377
mAP@0.5: 0.3073 | mAP@0.75: 0.1145
Avg predictions/image: 76.7 | High confidence (>0.5): 50.0%
Learning rate: 0.000020
No improvement for 4 epochs
Current: mAP@0.5=0.3073 | Best: 0.3077 (epoch 9)
```

```
=====
EPOCH 14/50
=====
Training Epoch 14: 100%|██████████| 2283/2283 [28:08<00:00, 1.35it/s, loss=1.43]
Training Loss: 1.0377
Validating Epoch 14: 100%|██████████| 763/763 [08:15<00:00, 1.54it/s, loss=1.47]
Validation Loss: 1.1374
mAP@0.5: 0.3076 | mAP@0.75: 0.1149
Avg predictions/image: 76.7 | High confidence (>0.5): 50.0%
Learning rate: 0.000020
No improvement for 5 epochs
Current: mAP@0.5=0.3076 | Best: 0.3077 (epoch 9)
```

```
=====
EPOCH 15/50
=====
Training Epoch 15: 100%|██████████| 2283/2283 [28:08<00:00, 1.35it/s, loss=1.4]
Training Loss: 1.0415
Validating Epoch 15: 100%|██████████| 763/763 [08:15<00:00, 1.54it/s, loss=1.51]
Validation Loss: 1.1367
mAP@0.5: 0.3076 | mAP@0.75: 0.1148
Avg predictions/image: 76.9 | High confidence (>0.5): 50.1%
Learning rate: 0.000002
No improvement for 6 epochs
Current: mAP@0.5=0.3076 | Best: 0.3077 (epoch 9)
```

```
=====
EPOCH 16/50
=====
Training Epoch 16: 100%|██████████| 2283/2283 [28:08<00:00, 1.35it/s, loss=0.874]
Training Loss: 1.0397
Validating Epoch 16: 100%|██████████| 763/763 [08:14<00:00, 1.54it/s, loss=1.45]
Validation Loss: 1.1368
mAP@0.5: 0.3075 | mAP@0.75: 0.1149
Avg predictions/image: 76.9 | High confidence (>0.5): 50.0%
Learning rate: 0.000002
No improvement for 7 epochs
Current: mAP@0.5=0.3075 | Best: 0.3077 (epoch 9)
```

```

=====
EPOCH 17/50
=====
Training Epoch 17: 100%|██████████| 2283/2283 [28:07<00:00, 1.35it/s, loss=1.51]
  Training Loss: 1.0404
Validating Epoch 17: 100%|██████████| 763/763 [08:14<00:00, 1.54it/s, loss=1.49]
  Validation Loss: 1.1369
  mAP@0.5: 0.3075 | mAP@0.75: 0.1148
  Avg predictions/image: 76.8 | High confidence (>0.5): 50.0%
  Learning rate: 0.000002
  No improvement for 8 epochs
  Current: mAP@0.5=0.3075 | Best: 0.3077 (epoch 9)

Early stopping triggered at epoch 17
  Final best mAP@0.5: 0.3077 (epoch 9)
Training history saved to: /content/drive/MyDrive/Building Damage Assessment/Object Detection Models/RV2(1)_training_history.json
"""

```

```

def get_modelV2(num_classes=2):
    weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = get_modelV2(num_classes=2)
model.to(device)

optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4, # Lower learning rate
    weight_decay=1e-4, # Lower weight decay
    amsgrad=True
)

# Use OneCycleLR for better convergence
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=5e-4,
    epochs=5,
    steps_per_epoch=len(train_loader),
    pct_start=0.3, # 30% of training for warmup
)

# Create model save directory
model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path = os.path.join(model_dir, 'ModelV2(2).pth')

# Start realistic training
history = main_training(
    model=model,
    train_loader=train_loader,

```

```

val_loader=val_loader,
optimizer=optimizer,
scheduler=scheduler,
device=device,
num_epochs=50,
patience=8,
model_save_path=model_save_path
)

history_save_path = os.path.join(model_dir, 'RV2(2)_training_history.json')
with open(history_save_path, 'w') as f:
    json.dump(history, f, indent=2)
print(f" Training history saved to: {history_save_path}")

```



Training (patience=8)

```

=====
EPOCH 1/50
=====
Training Epoch 1: 100%|██████████| 2283/2283 [08:08<00:00, 4.67it/s, loss=0.825]
  Training Loss: 1.2602
Validating Epoch 1: 100%|██████████| 763/763 [02:11<00:00, 5.81it/s, loss=1.61]
  Validation Loss: 1.1964
  mAP@0.5: 0.2720 | mAP@0.75: 0.1071
  Avg predictions/image: 78.3 | High confidence (>0.5): 47.3%
  Learning rate: 0.000020
Model improved! Saved at epoch 1
Best mAP@0.5: 0.2720 | Best mAP@0.75: 0.1071

=====
EPOCH 2/50
=====
Training Epoch 2: 100%|██████████| 2283/2283 [05:20<00:00, 7.11it/s, loss=0.898]
  Training Loss: 1.1019
Validating Epoch 2: 100%|██████████| 763/763 [01:53<00:00, 6.73it/s, loss=1.54]
  Validation Loss: 1.1378
  mAP@0.5: 0.3075 | mAP@0.75: 0.1197
  Avg predictions/image: 79.9 | High confidence (>0.5): 51.6%
  Learning rate: 0.000020
Model improved! Saved at epoch 2
Best mAP@0.5: 0.3075 | Best mAP@0.75: 0.1197

=====
EPOCH 3/50
=====
Training Epoch 3: 100%|██████████| 2283/2283 [05:08<00:00, 7.41it/s, loss=0.769]
  Training Loss: 1.0410
Validating Epoch 3: 100%|██████████| 763/763 [01:36<00:00, 7.90it/s, loss=1.47]
  Validation Loss: 1.1199
  mAP@0.5: 0.3097 | mAP@0.75: 0.1340
  Avg predictions/image: 75.2 | High confidence (>0.5): 54.9%
  Learning rate: 0.000020
Model improved! Saved at epoch 3
Best mAP@0.5: 0.3097 | Best mAP@0.75: 0.1340

=====
EPOCH 4/50
=====
Training Epoch 4: 100%|██████████| 2283/2283 [05:05<00:00, 7.48it/s, loss=0.888]

```

```

Training Loss: 0.9969
Validating Epoch 4: 100%|██████████| 763/763 [01:34<00:00, 8.03it/s, loss=1.41]
Validation Loss: 1.1244
mAP@0.5: 0.3278 | mAP@0.75: 0.1280
Avg predictions/image: 74.9 | High confidence (>0.5): 54.7%
Learning rate: 0.000020
Model improved! Saved at epoch 4
Best mAP@0.5: 0.3278 | Best mAP@0.75: 0.1280

=====
EPOCH 5/50

```

✓ FCOS

```

from torchvision.models.detection import fcos_resnet50_fpn, FCOS_ResNet50_FPN_Weights

def get_fcos_model(num_classes=2):
    # First create model with your desired num_classes (no pretrained weights)
    model = fcos_resnet50_fpn(weights=None, num_classes=num_classes)

    # Load a pretrained model to copy backbone weights from
    pretrained_model = fcos_resnet50_fpn(weights=FCOS_ResNet50_FPN_Weights.DEFAULT)

    # Copy only the backbone weights (FPN is integrated into backbone in FCOS)
    model.backbone.load_state_dict(pretrained_model.backbone.state_dict())

    # The head will be randomly initialized for your 2 classes
    return model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = get_fcos_model(num_classes=2)
model.to(device)

optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=0.002, # Lower learning rate
    momentum=0.9,
    weight_decay=0.0005
)

# Learning rate scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

# Save directory
model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path = os.path.join(model_dir, 'Model_FCOS1.pth')

history = main_training(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    scheduler=lr_scheduler,

```

```

    device=device,
    num_epochs=50,
    patience=8,
    model_save_path=model_save_path
)

```

"""FCOS Output:

Training (patience=8)

```

=====
EPOCH 1/50
=====
Training Epoch 1: 100%|██████████| 2283/2283 [19:28<00:00, 1.95it/s, loss=1.36]
  Training Loss: 1.5769
Validating Epoch 1: 100%|██████████| 763/763 [05:51<00:00, 2.17it/s, loss=1.4]
  Validation Loss: 1.4484
  mAP@0.5: 0.1770 | mAP@0.75: 0.0236
  Avg predictions/image: 100.0 | High confidence (>0.5): 33.2%
  Learning rate: 0.002000
Model improved! Saved at epoch 1
Best mAP@0.5: 0.1770 | Best mAP@0.75: 0.0236

```

```

=====
EPOCH 2/50
=====
Training Epoch 2: 100%|██████████| 2283/2283 [18:46<00:00, 2.03it/s, loss=1.35]
  Training Loss: 1.4026
Validating Epoch 2: 100%|██████████| 763/763 [05:33<00:00, 2.29it/s, loss=1.32]
  Validation Loss: 1.3701
  mAP@0.5: 0.2152 | mAP@0.75: 0.0480
  Avg predictions/image: 100.0 | High confidence (>0.5): 31.1%
  Learning rate: 0.002000
Model improved! Saved at epoch 2
Best mAP@0.5: 0.2152 | Best mAP@0.75: 0.0480

```

```

=====
EPOCH 3/50
=====
Training Epoch 3: 100%|██████████| 2283/2283 [18:46<00:00, 2.03it/s, loss=1.52]
  Training Loss: 1.3499
Validating Epoch 3: 100%|██████████| 763/763 [05:33<00:00, 2.29it/s, loss=1.29]
  Validation Loss: 1.3283
  mAP@0.5: 0.2481 | mAP@0.75: 0.0540
  Avg predictions/image: 99.7 | High confidence (>0.5): 32.4%
  Learning rate: 0.002000
Model improved! Saved at epoch 3
Best mAP@0.5: 0.2481 | Best mAP@0.75: 0.0540

```

```

=====
EPOCH 4/50
=====
Training Epoch 4: 100%|██████████| 2283/2283 [19:16<00:00, 1.97it/s, loss=1.27]
  Training Loss: 1.3222
Validating Epoch 4: 100%|██████████| 763/763 [05:42<00:00, 2.23it/s, loss=1.27]
  Validation Loss: 1.3045

```

```
mAP@0.5: 0.2602 | mAP@0.75: 0.0756
Avg predictions/image: 98.6 | High confidence (>0.5): 28.0%
Learning rate: 0.002000
Model improved! Saved at epoch 4
Best mAP@0.5: 0.2602 | Best mAP@0.75: 0.0756

=====
EPOCH 5/50
=====
Training Epoch 5: 100%|██████████| 2283/2283 [19:29<00:00, 1.95it/s, loss=1.68]
Training Loss: 1.3011
Validating Epoch 5: 100%|██████████| 763/763 [05:49<00:00, 2.18it/s, loss=1.27]
Validation Loss: 1.3008
mAP@0.5: 0.2631 | mAP@0.75: 0.0599
Avg predictions/image: 98.3 | High confidence (>0.5): 33.8%
Learning rate: 0.000200
Model improved! Saved at epoch 5
Best mAP@0.5: 0.2631 | Best mAP@0.75: 0.0599

=====
EPOCH 6/50
=====
Training Epoch 6: 100%|██████████| 2283/2283 [19:35<00:00, 1.94it/s, loss=1.25]
Training Loss: 1.2713
Validating Epoch 6: 100%|██████████| 763/763 [05:48<00:00, 2.19it/s, loss=1.25]
Validation Loss: 1.2789
mAP@0.5: 0.2792 | mAP@0.75: 0.0845
Avg predictions/image: 98.5 | High confidence (>0.5): 33.0%
Learning rate: 0.000200
Model improved! Saved at epoch 6
Best mAP@0.5: 0.2792 | Best mAP@0.75: 0.0845

=====
EPOCH 7/50
=====
Training Epoch 7: 100%|██████████| 2283/2283 [19:43<00:00, 1.93it/s, loss=1.31]
Training Loss: 1.2657
Validating Epoch 7: 100%|██████████| 763/763 [05:52<00:00, 2.16it/s, loss=1.25]
Validation Loss: 1.2749
mAP@0.5: 0.2821 | mAP@0.75: 0.0867
Avg predictions/image: 97.7 | High confidence (>0.5): 32.8%
Learning rate: 0.000200
Model improved! Saved at epoch 7
Best mAP@0.5: 0.2821 | Best mAP@0.75: 0.0867

=====
EPOCH 8/50
=====
Training Epoch 8: 100%|██████████| 2283/2283 [19:53<00:00, 1.91it/s, loss=1.39]
Training Loss: 1.2676
Validating Epoch 8: 100%|██████████| 763/763 [05:54<00:00, 2.15it/s, loss=1.25]
Validation Loss: 1.2750
mAP@0.5: 0.2834 | mAP@0.75: 0.0854
Avg predictions/image: 98.1 | High confidence (>0.5): 32.3%
Learning rate: 0.000200
Model improved! Saved at epoch 8
```


Best mAP@0.5: 0.2834 | Best mAP@0.75: 0.0854

=====

EPOCH 9/50

=====

Training Epoch 9: 100%|██████████| 2283/2283 [19:51<00:00, 1.92it/s, loss=1.23]

Training Loss: 1.2636

Validating Epoch 9: 100%|██████████| 763/763 [05:54<00:00, 2.15it/s, loss=1.25]

Validation Loss: 1.2722

mAP@0.5: 0.2861 | mAP@0.75: 0.0872

Avg predictions/image: 97.6 | High confidence (>0.5): 33.9%

Learning rate: 0.000200

Model improved! Saved at epoch 9

Best mAP@0.5: 0.2861 | Best mAP@0.75: 0.0872

=====

EPOCH 10/50

=====

Training Epoch 10: 100%|██████████| 2283/2283 [19:52<00:00, 1.91it/s, loss=1.3]

Training Loss: 1.2601

Validating Epoch 10: 100%|██████████| 763/763 [05:54<00:00, 2.15it/s, loss=1.25]

Validation Loss: 1.2698

mAP@0.5: 0.2869 | mAP@0.75: 0.0887

Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%

Learning rate: 0.000200

Model improved! Saved at epoch 10

Best mAP@0.5: 0.2869 | Best mAP@0.75: 0.0887

=====

EPOCH 11/50

=====

Training Epoch 11: 100%|██████████| 2283/2283 [19:51<00:00, 1.92it/s, loss=1.23]

Training Loss: 1.2586

Validating Epoch 11: 100%|██████████| 763/763 [05:53<00:00, 2.16it/s, loss=1.25]

Validation Loss: 1.2687

mAP@0.5: 0.2867 | mAP@0.75: 0.0897

Avg predictions/image: 96.9 | High confidence (>0.5): 33.2%

Learning rate: 0.000200

No improvement for 1 epochs

Current: mAP@0.5=0.2867 | Best: 0.2869 (epoch 10)

=====

EPOCH 12/50

=====

Training Epoch 12: 100%|██████████| 2283/2283 [19:53<00:00, 1.91it/s, loss=1.26]

Training Loss: 1.2575

Validating Epoch 12: 100%|██████████| 763/763 [05:50<00:00, 2.18it/s, loss=1.25]

Validation Loss: 1.2685

mAP@0.5: 0.2874 | mAP@0.75: 0.0899

Avg predictions/image: 96.8 | High confidence (>0.5): 33.4%

Learning rate: 0.000200

Model improved! Saved at epoch 12

Best mAP@0.5: 0.2874 | Best mAP@0.75: 0.0899

=====

EPOCH 13/50

```
=====
Training Epoch 13: 100%|██████████| 2283/2283 [19:37<00:00, 1.94it/s, loss=1.3]
Training Loss: 1.2585
Validating Epoch 13: 100%|██████████| 763/763 [05:50<00:00, 2.18it/s, loss=1.25]
Validation Loss: 1.2683
mAP@0.5: 0.2874 | mAP@0.75: 0.0908
Avg predictions/image: 96.7 | High confidence (>0.5): 33.0%
Learning rate: 0.000020
Model improved! Saved at epoch 13
Best mAP@0.5: 0.2874 | Best mAP@0.75: 0.0908
```

```
=====
EPOCH 14/50
```

```
=====
Training Epoch 14: 100%|██████████| 2283/2283 [19:45<00:00, 1.93it/s, loss=1.19]
Training Loss: 1.2582
Validating Epoch 14: 100%|██████████| 763/763 [05:49<00:00, 2.19it/s, loss=1.25]
Validation Loss: 1.2681
mAP@0.5: 0.2876 | mAP@0.75: 0.0903
Avg predictions/image: 96.6 | High confidence (>0.5): 32.8%
Learning rate: 0.000020
Model improved! Saved at epoch 14
Best mAP@0.5: 0.2876 | Best mAP@0.75: 0.0903
```

```
=====
EPOCH 15/50
```

```
=====
Training Epoch 15: 100%|██████████| 2283/2283 [19:33<00:00, 1.95it/s, loss=1.34]
Training Loss: 1.2599
Validating Epoch 15: 100%|██████████| 763/763 [05:50<00:00, 2.18it/s, loss=1.25]
Validation Loss: 1.2681
mAP@0.5: 0.2877 | mAP@0.75: 0.0907
Avg predictions/image: 96.4 | High confidence (>0.5): 32.6%
Learning rate: 0.000002
Model improved! Saved at epoch 15
Best mAP@0.5: 0.2877 | Best mAP@0.75: 0.0907
```

```
=====
EPOCH 16/50
```

```
=====
Training Epoch 16: 100%|██████████| 2283/2283 [19:32<00:00, 1.95it/s, loss=1.19]
Training Loss: 1.2585
Validating Epoch 16: 100%|██████████| 763/763 [05:49<00:00, 2.19it/s, loss=1.25]
Validation Loss: 1.2680
mAP@0.5: 0.2878 | mAP@0.75: 0.0904
Avg predictions/image: 96.7 | High confidence (>0.5): 32.9%
Learning rate: 0.000002
Model improved! Saved at epoch 16
Best mAP@0.5: 0.2878 | Best mAP@0.75: 0.0904
```

```
=====
EPOCH 17/50
```

```
=====
Training Epoch 17: 100%|██████████| 2283/2283 [19:33<00:00, 1.95it/s, loss=1.29]
Training Loss: 1.2577
Validating Epoch 17: 100%|██████████| 763/763 [05:49<00:00, 2.18it/s, loss=1.25]
```

```
Validation Loss: 1.2679
mAP@0.5: 0.2877 | mAP@0.75: 0.0904
Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%
Learning rate: 0.000002
No improvement for 1 epochs
Current: mAP@0.5=0.2877 | Best: 0.2878 (epoch 16)

=====
EPOCH 18/50
=====
Training Epoch 18: 100%|██████████| 2283/2283 [19:33<00:00, 1.95it/s, loss=1.08]
Training Loss: 1.2570
Validating Epoch 18: 100%|██████████| 763/763 [05:48<00:00, 2.19it/s, loss=1.25]
Validation Loss: 1.2679
mAP@0.5: 0.2878 | mAP@0.75: 0.0903
Avg predictions/image: 96.6 | High confidence (>0.5): 32.9%
Learning rate: 0.000002
No improvement for 2 epochs
Current: mAP@0.5=0.2878 | Best: 0.2878 (epoch 16)

=====
EPOCH 19/50
=====
Training Epoch 19: 100%|██████████| 2283/2283 [19:29<00:00, 1.95it/s, loss=1.17]
Training Loss: 1.2576
Validating Epoch 19: 100%|██████████| 763/763 [05:46<00:00, 2.20it/s, loss=1.25]
Validation Loss: 1.2679
mAP@0.5: 0.2876 | mAP@0.75: 0.0904
Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%
Learning rate: 0.000002
No improvement for 3 epochs
Current: mAP@0.5=0.2876 | Best: 0.2878 (epoch 16)

=====
EPOCH 20/50
=====
Training Epoch 20: 100%|██████████| 2283/2283 [19:30<00:00, 1.95it/s, loss=1.27]
Training Loss: 1.2564
Validating Epoch 20: 100%|██████████| 763/763 [05:46<00:00, 2.20it/s, loss=1.25]
Validation Loss: 1.2678
mAP@0.5: 0.2876 | mAP@0.75: 0.0905
Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%
Learning rate: 0.000000
No improvement for 4 epochs
Current: mAP@0.5=0.2876 | Best: 0.2878 (epoch 16)

=====
EPOCH 21/50
=====
Training Epoch 21: 100%|██████████| 2283/2283 [19:31<00:00, 1.95it/s, loss=1.21]
Training Loss: 1.2562
Validating Epoch 21: 100%|██████████| 763/763 [05:52<00:00, 2.17it/s, loss=1.25]
Validation Loss: 1.2678
mAP@0.5: 0.2877 | mAP@0.75: 0.0905
Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%
Learning rate: 0.000000
```

```

No improvement for 5 epochs
Current: mAP@0.5=0.2877 | Best: 0.2878 (epoch 16)

=====
EPOCH 22/50
=====
Training Epoch 22: 100%|██████████| 2283/2283 [19:31<00:00, 1.95it/s, loss=1.38]
  Training Loss: 1.2575
Validating Epoch 22: 100%|██████████| 763/763 [05:48<00:00, 2.19it/s, loss=1.25]
  Validation Loss: 1.2678
  mAP@0.5: 0.2876 | mAP@0.75: 0.0905
  Avg predictions/image: 96.5 | High confidence (>0.5): 32.8%
  Learning rate: 0.000000
No improvement for 6 epochs
Current: mAP@0.5=0.2876 | Best: 0.2878 (epoch 16)

=====
EPOCH 23/50
=====
Training Epoch 23: 100%|██████████| 2283/2283 [19:27<00:00, 1.96it/s, loss=1.42]
  Training Loss: 1.2584
Validating Epoch 23: 100%|██████████| 763/763 [05:47<00:00, 2.20it/s, loss=1.25]
  Validation Loss: 1.2678
  mAP@0.5: 0.2876 | mAP@0.75: 0.0904
  Avg predictions/image: 96.5 | High confidence (>0.5): 32.9%
  Learning rate: 0.000000
No improvement for 7 epochs
Current: mAP@0.5=0.2876 | Best: 0.2878 (epoch 16)

=====
EPOCH 24/50
=====
Training Epoch 24: 100%|██████████| 2283/2283 [19:32<00:00, 1.95it/s, loss=1.27]
  Training Loss: 1.2575
Validating Epoch 24: 100%|██████████| 763/763 [05:48<00:00, 2.19it/s, loss=1.25]
  Validation Loss: 1.2678
  mAP@0.5: 0.2876 | mAP@0.75: 0.0904
  Avg predictions/image: 96.5 | High confidence (>0.5): 32.9%
  Learning rate: 0.000000
No improvement for 8 epochs
Current: mAP@0.5=0.2876 | Best: 0.2878 (epoch 16)

Early stopping triggered at epoch 24
  Final best mAP@0.5: 0.2878 (epoch 16)
"""

# Save training history
history_save_path = os.path.join(model_dir, 'FCOS1_training_history.json')
with open(history_save_path, 'w') as f:
    json.dump(history, f, indent=2)
print(f" Training history saved to: {history_save_path}")

```

✓ Further Fine Tuning (ResNet50 V2)

```

def get_transformsV2(train=True):
    """Fixed version - no warnings"""
    if train:
        return A.Compose([
            # Basic geometric
            A.HorizontalFlip(p=0.5),
            A.Rotate(limit=10, p=0.3),

            # Use Affine instead of ShiftScaleRotate (fixes warning 1)
            A.Affine(
                scale=(0.9, 1.1),      # 10% scale variation
                translate_percent=(-0.05, 0.05), # 5% translation
                rotate=(-5, 5),        # 5 degree rotation
                shear=(-2, 2),          # Small shear
                p=0.4
            ),

            # Color augmentations
            A.RandomBrightnessContrast(
                brightness_limit=0.15,
                contrast_limit=0.15,
                p=0.4
            ),
            A.HueSaturationValue(
                hue_shift_limit=5,
                sat_shift_limit=15,
                val_shift_limit=10,
                p=0.3
            ),

            # Fixed blur limits (minimum 3)
            A.OneOf([
                A.Blur(blur_limit=3, p=1.0),
                A.GaussianBlur(blur_limit=3, p=1.0),
            ], p=0.15),

            A.Resize(640, 640),
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2()
        ], bbox_params=A.BboxParams(
            format='pascal_voc',
            label_fields=['labels'],
            min_visibility=0.25,
            min_area=20.0,
            check_each_transform=True
        ))
    else:
        return A.Compose([
            A.Resize(640, 640),
            A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
            ToTensorV2()
        ], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

```

```
# Create datasets (when using Google Colab)
train_dataset_fixed = BuildingDataset(
    '/content/tier1/images',
    '/content/drive/MyDrive/Building Damage Assessment/Annotations/tier1_annotations.json',
    transforms=get_transformsV2(train=True)
)

val_dataset_fixed = BuildingDataset(
    '/content/holdout/images',
    '/content/drive/MyDrive/Building Damage Assessment/Annotations/holdout_annotations.json',
    transforms=get_transformsV2(train=False)
)

print(f"Train dataset size: {len(train_dataset_fixed)}")
print(f"Validation dataset size: {len(val_dataset_fixed)}")
```

 Loaded 4566 images
Loaded 1526 images
Train dataset size: 4566
Validation dataset size: 1526

```
# Create datasets (when running locally)
train_dataset_fixed = BuildingDataset(
    r'C:\Users\sajja\Desktop\Dissertation\geotiffs\tier1\images',
    r'C:\Users\sajja\Desktop\Dissertation\Annotations\tier1_annotations.json',
    transforms=get_transformsV2(train=True)
)

val_dataset_fixed = BuildingDataset(
    r'C:\Users\sajja\Desktop\Dissertation\geotiffs\hold\images',
    r'C:\Users\sajja\Desktop\Dissertation\Annotations\holdout_annotations.json',
    transforms=get_transformsV2(train=False)
)

print(f"Train dataset size: {len(train_dataset_fixed)}")
print(f"Validation dataset size: {len(val_dataset_fixed)}")
```

 Loaded 4566 images
Loaded 1526 images
Train dataset size: 4566
Validation dataset size: 1526

```
# Create data loaders with smaller batch size
train_loader = DataLoader(
    train_dataset_fixed,
    batch_size=4, # Keep small for memory
    shuffle=True,
    collate_fn=collate_fn,
    num_workers=2
)

val_loader = DataLoader(
    val_dataset_fixed,
    batch_size=4,
    shuffle=False,
    collate_fn=collate_fn,
```

```

    num_workers=2
)

```

▼ Stage 1

```

import torch
import torch.nn as nn
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
import os
import json

def get_modelV2(num_classes=2):
    weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

# A partial freezing function to freeze early layers of the backbone
# conv1, bn1, layer1, layer2 are frozen, while layer3, layer4, FPN, RPN, and ROI heads remain trainable
def freeze_backbone_partial(model):
    backbone = model.backbone.body

    # Freeze early layers
    for name, param in backbone.named_parameters():
        if any(layer in name for layer in ['conv1', 'bn1', 'layer1', 'layer2']):
            param.requires_grad = False
        else: # layer3, layer4 remain trainable
            param.requires_grad = True

    # Keep FPN trainable
    for param in model.backbone.fpn.parameters():
        param.requires_grad = True

    # Keep RPN trainable
    for param in model.rpn.parameters():
        param.requires_grad = True

    # Keep ROI heads trainable
    for param in model.roi_heads.parameters():
        param.requires_grad = True

def count_trainable_parameters(model):
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    frozen = total - trainable

    print(f"\nParameter Summary:")
    print(f"Total parameters: {total:,}")
    print(f"Trainable parameters: {trainable:,} ({100*trainable/total:.1f}%)")
    print(f"Frozen parameters: {frozen:,} ({100*frozen/total:.1f}%)")
    return trainable, total

```

```

# Training setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model = get_modelV2(num_classes=2)
model.to(device)

current_model_path = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models/ModelV2(1).pth'
#current_model_path = r'C:\Users\sajja\Desktop\Dissertation\Object Detection Models\ModelV2(1).pth'

if os.path.exists(current_model_path):
    print(f"\n Loading best model from: {current_model_path}")
    model.load_state_dict(torch.load(current_model_path, map_location=device))

# Apply partial freezing
freeze_backbone_partial(model)
count_trainable_parameters(model)

# Setup optimizer with lower learning rate for fine-tuning
optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4, # Lower learning rate
    weight_decay=1e-4, # Lower weight decay
    amsgrad=True
)

# Lower learning rate scheduler
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.3)

# Setup save paths
model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path_s1 = os.path.join(model_dir, 'ModelV2_Stage1.pth')

history_s1 = main_training(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    scheduler=lr_scheduler,
    device=device,
    num_epochs=15,
    patience=6,
    model_save_path=model_save_path_s1
)

# Training history
history_s1_path = os.path.join(model_dir, 'Stage1_training_history.json')
with open(history_s1_path, 'w') as f:
    json.dump(history_s1, f, indent=2)

print(f" Best model saved to: {model_save_path_s1}")
print(f" Training history saved to: {history_s1_path}")

```

🔄 Using device: cuda

Loading best model from: /content/drive/MyDrive/Building Damage Assessment/Object Detection Models/ModelV2(1).pth

Parameter Summary:

Total parameters: 43,256,153
 Trainable parameters: 37,741,849 (87.3%)
 Frozen parameters: 5,514,304 (12.7%)

Training (patience=6)

```
=====
EPOCH 1/15
=====
Training Epoch 1: 100%|██████████| 2283/2283 [25:51<00:00, 1.47it/s, loss=1.15]
  Training Loss: 1.1315
Validating Epoch 1: 100%|██████████| 763/763 [07:50<00:00, 1.62it/s, loss=1.6]
  Validation Loss: 1.1564
  mAP@0.5: 0.2873 | mAP@0.75: 0.1048
  Avg predictions/image: 79.8 | High confidence (>0.5): 48.7%
  Learning rate: 0.000500
  Model improved! Saved at epoch 1
  Best mAP@0.5: 0.2873 | Best mAP@0.75: 0.1048

=====
EPOCH 2/15
=====
Training Epoch 2: 100%|██████████| 2283/2283 [25:18<00:00, 1.50it/s, loss=1.32]
  Training Loss: 1.1207
Validating Epoch 2: 100%|██████████| 763/763 [07:47<00:00, 1.63it/s, loss=1.56]
  Validation Loss: 1.1561
  mAP@0.5: 0.2887 | mAP@0.75: 0.1065
  Avg predictions/image: 77.8 | High confidence (>0.5): 49.5%
  Learning rate: 0.000500
  Model improved! Saved at epoch 2
  Best mAP@0.5: 0.2887 | Best mAP@0.75: 0.1065

=====
EPOCH 3/15
=====
Training Epoch 3: 100%|██████████| 2283/2283 [25:30<00:00, 1.49it/s, loss=0.878]
  Training Loss: 1.1140
Validating Epoch 3: 100%|██████████| 763/763 [07:48<00:00, 1.63it/s, loss=1.61]
  Validation Loss: 1.1591
  mAP@0.5: 0.2858 | mAP@0.75: 0.1045
  Avg predictions/image: 78.9 | High confidence (>0.5): 48.9%
  Learning rate: 0.000500
  No improvement for 1 epochs
  Current: mAP@0.5=0.2858 | Best: 0.2887 (epoch 2)

=====
EPOCH 4/15
=====
Training Epoch 4: 100%|██████████| 2283/2283 [25:28<00:00, 1.49it/s, loss=0.842]
  Training Loss: 1.1080
Validating Epoch 4: 100%|██████████| 763/763 [07:51<00:00, 1.62it/s, loss=1.54]
  Validation Loss: 1.1642
  mAP@0.5: 0.2774 | mAP@0.75: 0.1018
```

▼ Stage 2

```
import torch
import torch.nn as nn
```

```

from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
import os
import json

def get_modelV2(num_classes=2):
    weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

# Full unfreezing function - unfreeze all layers
def unfreeze_all_layers(model):
    for param in model.parameters():
        param.requires_grad = True
    print("All layers are now trainable")

def count_trainable_parameters(model):
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    frozen = total - trainable

    print(f"\nParameter Summary:")
    print(f"Total parameters: {total:,}")
    print(f"Trainable parameters: {trainable:,} ({100*trainable/total:.1f}%)")
    print(f"Frozen parameters: {frozen:,} ({100*frozen/total:.1f}%)")
    return trainable, total

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model = get_modelV2(num_classes=2)
model.to(device)

# Load Stage 1 best model
stage1_model_path = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models/ModelV2_Stage1.pth'
#stage1_model_path = r'C:\Users\sajja\Desktop\Dissertation\Object Detection Models\ModelV2_Stage1.pth'

if os.path.exists(stage1_model_path):
    model.load_state_dict(torch.load(stage1_model_path, map_location=device))
    print("Stage 1 model loaded")

# Apply full unfreezing
unfreeze_all_layers(model)
count_trainable_parameters(model)

# Setup optimizer with very low learning rate for full unfreezing
optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4, # Lower learning rate
    weight_decay=1e-4, # Lower weight decay
    amsgrad=True
)

# Conservative learning rate scheduler

```

```

lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)

# Setup save paths
model_dir = '/content/drive/MyDrive/Building Damage Assessment/Object Detection Models'
os.makedirs(model_dir, exist_ok=True)
model_save_path_s2 = os.path.join(model_dir, 'ModelV2_Stage2_Final.pth')

history_s2 = main_training(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    scheduler=lr_scheduler,
    device=device,
    num_epochs=15,
    patience=5,
    model_save_path=model_save_path_s2
)

# Save training history
history_s2_path = os.path.join(model_dir, 'Stage2_training_history.json')
with open(history_s2_path, 'w') as f:
    json.dump(history_s2, f, indent=2)

print(f" Final model saved to: {model_save_path_s2}")
print(f" Training history saved to: {history_s2_path}")

```



Using device: cuda
 Stage 1 model loaded
 All layers are now trainable

Parameter Summary:
 Total parameters: 43,256,153
 Trainable parameters: 43,256,153 (100.0%)
 Frozen parameters: 0 (0.0%)

Training (patience=5)

```

=====
EPOCH 1/15
=====
Training Epoch 1: 100%|██████████| 2283/2283 [31:34<00:00, 1.20it/s, loss=1.08]
Training Loss: 1.1012
Validating Epoch 1: 100%|██████████| 763/763 [08:20<00:00, 1.53it/s, loss=1.67]
Validation Loss: 1.1543
mAP@0.5: 0.2866 | mAP@0.75: 0.1060
Avg predictions/image: 78.1 | High confidence (>0.5): 50.0%
Learning rate: 0.000100
Model improved! Saved at epoch 1
Best mAP@0.5: 0.2866 | Best mAP@0.75: 0.1060

=====
EPOCH 2/15
=====
Training Epoch 2: 100%|██████████| 2283/2283 [31:34<00:00, 1.21it/s, loss=0.599]
Training Loss: 1.1009
Validating Epoch 2: 100%|██████████| 763/763 [08:19<00:00, 1.53it/s, loss=1.67]
Validation Loss: 1.1513
mAP@0.5: 0.2872 | mAP@0.75: 0.1069

```

```

Avg predictions/image: 77.7 | High confidence (>0.5): 49.3%
Learning rate: 0.000100
Model improved! Saved at epoch 2
Best mAP@0.5: 0.2872 | Best mAP@0.75: 0.1069

```

```
=====
```

```
EPOCH 3/15
```

```
=====
```

```

Training Epoch 3: 100%|██████████| 2283/2283 [31:29<00:00, 1.21it/s, loss=0.409]
Training Loss: 1.0972
Validating Epoch 3: 100%|██████████| 763/763 [08:11<00:00, 1.55it/s, loss=1.56]
Validation Loss: 1.1530
mAP@0.5: 0.2876 | mAP@0.75: 0.1065
Avg predictions/image: 78.1 | High confidence (>0.5): 49.4%
Learning rate: 0.000100
Model improved! Saved at epoch 3
Best mAP@0.5: 0.2876 | Best mAP@0.75: 0.1065

```

```
=====
```

```
EPOCH 4/15
```

```
=====
```

```

Training Epoch 4: 100%|██████████| 2283/2283 [31:28<00:00, 1.21it/s, loss=1.28]
Training Loss: 1.0952
Validating Epoch 4: 100%|██████████| 763/763 [08:21<00:00, 1.52it/s, loss=1.55]
Validation Loss: 1.1531
mAP@0.5: 0.2852 | mAP@0.75: 0.1032

```

✓ YOLOv8

```
!pip install ultralytics
```



Show hidden output

```

# Extract Holdout (Validation)
holdout_base = '/content/drive/MyDrive/Building Damage Assessment/Holdout'
holdout_images_zip = os.path.join(holdout_base, 'images.zip')
holdout_labels_zip = os.path.join(holdout_base, 'labels.zip')

```

```

if os.path.exists(holdout_images_zip):
    with zipfile.ZipFile(holdout_images_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/holdout')
    print('Holdout images extracted')

```

```

if os.path.exists(holdout_labels_zip):
    with zipfile.ZipFile(holdout_labels_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/holdout_labels')
    print('Holdout labels extracted')

```



Holdout images extracted
Holdout labels extracted

```

from ultralytics import YOLO
import os
import glob
from PIL import Image

```

```
import cv2
import numpy as np
from tqdm import tqdm
```



Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

Analysing .tif images and converting to .png to be compatible with YOLOv8

```
def analyze_tiff_file(file_path):
    """Analyze a TIFF file to understand its properties"""
    print(f"\nAnalyzing: {os.path.basename(file_path)}")

    # Check file size
    file_size = os.path.getsize(file_path)
    print(f"File size: {file_size} bytes")

    # Try to read first few bytes
    try:
        with open(file_path, 'rb') as f:
            header = f.read(16)
            print(f"Header bytes: {header[:8].hex()}")

            # TIFF files should start with either:
            # II* (Intel/little-endian): 49 49 2A 00
            # MM* (Motorola/big-endian): 4D 4D 00 2A
            if header[:4] in [b'II*\x00', b'MM*\x00*']:
                print("Valid TIFF header detected")
            else:
                print("Invalid TIFF header - file may be corrupted")
    except Exception as e:
        print(f"Error reading file header: {e}")

def convert_with_opencv(input_path, output_path):
    """Try converting using OpenCV"""
    try:
        # Read with OpenCV
        img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)
        if img is None:
            return False, "OpenCV couldn't read the image"

        # Convert BGR to RGB if needed
        if len(img.shape) == 3 and img.shape[2] == 3:
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Save as PNG using OpenCV
        cv2.imwrite(output_path, img)
        return True, "Success with OpenCV"
    except Exception as e:
        return False, f"OpenCV error: {e}"

def convert_with_pillow_force(input_path, output_path):
    """Try converting with PIL using different modes"""
    methods = [
        lambda: Image.open(input_path),
        lambda: Image.open(input_path).convert('RGB'),
```

```

        lambda: Image.open(input_path).convert('RGBA'),
    ]

    for i, method in enumerate(methods):
        try:
            img = method()
            img.save(output_path, 'PNG')
            return True, f"Success with PIL method {i+1}"
        except Exception as e:
            continue

    return False, "All PIL methods failed"

def repair_and_convert_tiff(input_folder, output_folder, sample_file=None):
    """
    Robust TIFF to PNG conversion with multiple fallback methods
    """
    # Create output folder
    os.makedirs(output_folder, exist_ok=True)

    # Find all TIFF files
    tiff_patterns = ['*.tif', '*.tiff', '*.TIF', '*.TIFF']
    tiff_files = []

    for pattern in tiff_patterns:
        tiff_files.extend(glob.glob(os.path.join(input_folder, '**', pattern), recursive=True))

    print(f"Found {len(tiff_files)} TIFF files")

    # If sample_file is specified, analyze just that one
    if sample_file:
        sample_path = None
        for tiff_file in tiff_files:
            if sample_file in os.path.basename(tiff_file):
                sample_path = tiff_file
                break

        if sample_path:
            analyze_tiff_file(sample_path)
            return
        else:
            print(f"Sample file '{sample_file}' not found")
            return

    for i, tiff_file in enumerate(tiff_files[:3]):
        analyze_tiff_file(tiff_file)
        if i < 2: # Don't print separator after last file
            print("-" * 30)

    # Since files have valid headers, proceed with conversion
    print(f"\nTIFF files appear to have valid headers. Proceeding with conversion...")
    proceed = True

    # Attempt conversion with multiple methods
    converted_count = 0
    failed_files = []

```

```

for tiff_file in tqdm(tiff_files, desc="Converting files"):
    # Get relative path and create output path
    rel_path = os.path.relpath(tiff_file, input_folder)
    png_filename = os.path.splitext(rel_path)[0] + '.png'
    output_path = os.path.join(output_folder, png_filename)

    # Create output directory
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    # Try different conversion methods
    methods = [
        ("OpenCV", lambda: convert_with_opencv(tiff_file, output_path)),
        ("PIL Force", lambda: convert_with_pillow_force(tiff_file, output_path)),
    ]

    converted = False
    for method_name, method_func in methods:
        try:
            success, message = method_func()
            if success:
                converted_count += 1
                converted = True
                break
        except Exception as e:
            continue

    if not converted:
        failed_files.append(tiff_file)

print(f"\n\nConversion Results:")
print(f"Successfully converted: {converted_count}")
print(f"Failed: {len(failed_files)}")

if failed_files:
    print(f"\n\nFirst 5 failed files:")
    for failed_file in failed_files[:5]:
        print(f"  {os.path.basename(failed_file)}")

repair_and_convert_tiff(
    input_folder='/content/tier1/images',
    output_folder='/content/tier1_png'
)

```

🔗 Found 5598 TIFF files

```

Analyzing: hurricane-michael_00000162_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----

```

```

Analyzing: social-fire_00001343_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----

```

```
Analyzing: santa-rosa-wildfire_00000287_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
\nTIFF files appear to have valid headers. Proceeding with conversion...
Converting files: 100%|██████████| 5598/5598 [05:08<00:00, 18.14it/s]\nConversion Results:
Successfully converted: 5598
Failed: 0
```

```
repair_and_convert_tiff(
    input_folder='/content/holdout/images',
    output_folder='/content/holdout_png'
)
```

➦ Found 1866 TIFF files

```
Analyzing: santa-rosa-wildfire_00000332_post_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----
```

```
Analyzing: hurricane-florence_00000534_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----
```

```
Analyzing: hurricane-harvey_00000311_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
\nTIFF files appear to have valid headers. Proceeding with conversion...
Converting files: 100%|██████████| 1866/1866 [02:04<00:00, 14.94it/s]\nConversion Results:
Successfully converted: 1866
Failed: 0
```

```
base_model = "/content/drive/MyDrive/YOLO_Backups/building_detection_yolov8s/weights/best.pt"
best_model = "/content/drive/MyDrive/YOLO_Backups_final/building_detection_yolov8s_final/weights/best.pt"
```

```
# Set up of dataset (in .yaml) to be compatible with ultralytics (github: https://github.com/ultralytics/ultralytics)
```

```
def setup_yolo_dataset_with_labels():
```

```
    """Complete setup of YOLO dataset with converted labels"""
```

```
    print("="*60)
    print("SETTING UP YOLO DATASET WITH LABELS")
    print("="*60)
```

```
# Paths
```

```
PROCESSED_DATA = '/content/drive/MyDrive/Building Damage Assessment/Processed_Data'
YOLO_DATASET_ROOT = '/content/yolo_dataset'
```

```
# Create YOLO directory structure
```

```
os.makedirs(YOLO_DATASET_ROOT, exist_ok=True)
```



```

for split in ['train', 'val']:
    os.makedirs(f'{YOLO_DATASET_ROOT}/images/{split}', exist_ok=True)
    os.makedirs(f'{YOLO_DATASET_ROOT}/labels/{split}', exist_ok=True)

# Convert annotations for each split
splits_mapping = {
    'train': {
        'coco_file': f'{PROCESSED_DATA}/train/train_annotations.json',
        'labels_dir': f'{YOLO_DATASET_ROOT}/labels/train',
        'images_source': '/content/tier1_png',
        'images_dest': f'{YOLO_DATASET_ROOT}/images/train'
    },
    'val': {
        'coco_file': f'{PROCESSED_DATA}/val/val_annotations.json',
        'labels_dir': f'{YOLO_DATASET_ROOT}/labels/val',
        'images_source': '/content/holdout_png',
        'images_dest': f'{YOLO_DATASET_ROOT}/images/val'
    }
}

for split, paths in splits_mapping.items():
    print(f"\n{' '*30}")
    print(f"PROCESSING {split.upper()} SPLIT")
    print(f"{' '*30}")

    # Check if COCO annotation file exists
    if not os.path.exists(paths['coco_file']):
        print(f"ERROR: COCO annotation file not found: {paths['coco_file']}")
        continue

    # Convert COCO to YOLO
    converted_count = convert_coco_to_yolo(paths['coco_file'], paths['labels_dir'])

    # Link/copy images to YOLO structure
    if os.path.exists(paths['images_source']):
        import glob
        import shutil

        png_files = glob.glob(os.path.join(paths['images_source'], '**', '*.png'), recursive=True)
        print(f"Found {len(png_files)} PNG images to organize")

        linked_count = 0
        for png_file in tqdm(png_files[:10], desc=f"Organizing {split} images"): # Show first 10
            filename = os.path.basename(png_file)
            dest_path = os.path.join(paths['images_dest'], filename)

            try:
                if not os.path.exists(dest_path):
                    os.symlink(png_file, dest_path)
                linked_count += 1
            except:
                shutil.copy2(png_file, dest_path)
                linked_count += 1

        if len(png_files) > 10:
            # Link the rest without progress bar for speed

```

```

for png_file in png_files[10:]:
    filename = os.path.basename(png_file)
    dest_path = os.path.join(paths['images_dest'], filename)

    try:
        if not os.path.exists(dest_path):
            os.symlink(png_file, dest_path)
            linked_count += 1
    except:
        try:
            shutil.copy2(png_file, dest_path)
            linked_count += 1
        except:
            pass

    print(f"Organized {linked_count} images for {split}")
else:
    print(f"WARNING: Images source not found: {paths['images_source']}")

print("CREATING DATASET.YAML")

dataset_config = {
    'path': YOLO_DATASET_ROOT,
    'train': 'images/train',
    'val': 'images/val',
    'nc': 1, # number of classes
    'names': ['building'] # class names
}

import yaml
dataset_yaml_path = os.path.join(YOLO_DATASET_ROOT, 'dataset.yaml')
with open(dataset_yaml_path, 'w') as f:
    yaml.dump(dataset_config, f, default_flow_style=False)

print(f"Created dataset.yaml at: {dataset_yaml_path}")

print("VERIFYING YOLO DATASET")

for split in ['train', 'val']:
    images_dir = f'{YOLO_DATASET_ROOT}/images/{split}'
    labels_dir = f'{YOLO_DATASET_ROOT}/labels/{split}'

    if os.path.exists(images_dir) and os.path.exists(labels_dir):
        import glob
        images_count = len(glob.glob(os.path.join(images_dir, '*.png')))
        labels_count = len(glob.glob(os.path.join(labels_dir, '*.txt')))

        print(f"{split.upper()}:")
        print(f"  Images: {images_count}")
        print(f"  Labels: {labels_count}")

setup_yolo_dataset_with_labels()

model = YOLO('yolov8s.pt')

```

```
# Initial training of model with 30 epochs and resized images to 640
results = model.train(
    data='/content/yolo_dataset/dataset.yaml',
    epochs=30,
    imgsz=512,
    batch=4,
    patience=8,
    name='building_detection_yolov8s',
    project='/content/yolo_runs'
)
```

Results from 30 epochs training

```
# @title Results from 30 epochs training
""" Results from 30 epochs training
Ultralytics 8.3.170 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=4, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, conf=None, copy_paste=
Overriding model.yaml nc=80 with nc=1
```

	from	n	params	module	arguments
0	-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]
1	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
2	-1	1	29056	ultralytics.nn.modules.block.C2f	[64, 64, 1, True]
3	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
4	-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
5	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
6	-1	2	788480	ultralytics.nn.modules.block.C2f	[256, 256, 2, True]
7	-1	1	1180672	ultralytics.nn.modules.conv.Conv	[256, 512, 3, 2]
8	-1	1	1838080	ultralytics.nn.modules.block.C2f	[512, 512, 1, True]
9	-1	1	656896	ultralytics.nn.modules.block.SPPF	[512, 512, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12	-1	1	591360	ultralytics.nn.modules.block.C2f	[768, 256, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15	-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
16	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18	-1	1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
19	-1	1	590336	ultralytics.nn.modules.conv.Conv	[256, 256, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21	-1	1	1969152	ultralytics.nn.modules.block.C2f	[768, 512, 1]
22	[15, 18, 21]	1	2116435	ultralytics.nn.modules.head.Detect	[1, [128, 256, 512]]

Model summary: 129 layers, 11,135,987 parameters, 11,135,971 gradients, 28.6 GFLOPs

```
Transferred 349/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed
train: Fast image access (ping: 0.0±0.0 ms, read: 112.9±11.3 MB/s, size: 544.5 KB)
train: Scanning /content/yolo_dataset/labels/train... 5598 images, 1032 backgrounds, 0 corrupt: 100%| 5598/5598 [00:25<00:00, 223.74it/s]train: /content/yolo_dataset/images/train/palu-tsunami
train: /content/yolo_dataset/images/train/palu-tsunami_00000106_pre_disaster.png: 1 duplicate labels removed

train: New cache created: /content/yolo_dataset/labels/train.cache
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_gric
```

val: Fast image access  (ping: 0.0±0.0 ms, read: 504.4±934.6 MB/s, size: 558.2 KB)

val: Scanning /content/yolo_dataset/labels/val... 1866 images, 340 backgrounds, 0 corrupt: 100% 1866/1866 [00:08<00:00, 212.94it/s]val: New cache created: /content/yolo_dataset/labels/val.

Plotting labels to /content/yolo_runs/building_detection_yolov8s/labels.jpg...

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...

optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)

Image sizes 512 train, 512 val

Using 2 dataloader workers

Logging results to /content/yolo_runs/building_detection_yolov8s

Starting training for 30 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/30	7.04G	2.26	1.726	1.26	233	640: 100%  1400/1400 [03:45<00:00, 6.22it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:39<00:00, 5.96it/s]
	all	1866	108784	0.524	0.379	0.371 0.165
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/30	7.08G	2.177	1.488	1.235	351	640: 100%  1400/1400 [03:37<00:00, 6.44it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:38<00:00, 6.05it/s]
	all	1866	108784	0.497	0.372	0.344 0.15
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/30	7.12G	2.16	1.432	1.212	354	640: 100%  1400/1400 [03:40<00:00, 6.34it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:38<00:00, 6.07it/s]
	all	1866	108784	0.589	0.393	0.416 0.194
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
4/30	7.15G	2.122	1.383	1.2	87	640: 100%  1400/1400 [03:46<00:00, 6.19it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:38<00:00, 6.10it/s]
	all	1866	108784	0.606	0.416	0.437 0.205
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/30	7.19G	2.086	1.378	1.192	69	640: 100%  1400/1400 [03:39<00:00, 6.37it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:37<00:00, 6.19it/s]
	all	1866	108784	0.626	0.419	0.447 0.215
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
6/30	7.22G	2.057	1.314	1.178	134	640: 100%  1400/1400 [03:28<00:00, 6.71it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:36<00:00, 6.50it/s]
	all	1866	108784	0.615	0.426	0.452 0.218
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
7/30	7.26G	2.027	1.272	1.168	335	640: 100%  1400/1400 [03:28<00:00, 6.72it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:35<00:00, 6.68it/s]
	all	1866	108784	0.607	0.419	0.445 0.214
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
8/30	7.3G	2.034	1.269	1.168	140	640: 100%  1400/1400 [03:29<00:00, 6.69it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:35<00:00, 6.60it/s]
	all	1866	108784	0.631	0.43	0.463 0.228
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
9/30	7.33G	2.004	1.253	1.154	31	640: 100%  1400/1400 [03:30<00:00, 6.66it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:35<00:00, 6.52it/s]
	all	1866	108784	0.64	0.441	0.472 0.235

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
10/30	7.37G	2.001	1.22	1.152	421	640: 100% ██████████ 1400/1400 [03:29<00:00, 6.69it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:35<00:00, 6.52it/s]
	all	1866	108784	0.639	0.441	0.476 0.237
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
11/30	7.66G	1.985	1.212	1.149	160	640: 100% ██████████ 1400/1400 [03:30<00:00, 6.66it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:36<00:00, 6.47it/s]
	all	1866	108784	0.657	0.444	0.488 0.243
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
12/30	6.38G	1.973	1.21	1.147	372	640: 100% ██████████ 1400/1400 [03:28<00:00, 6.72it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:36<00:00, 6.46it/s]
	all	1866	108784	0.647	0.434	0.475 0.234
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
13/30	7.5G	1.961	1.197	1.135	30	640: 100% ██████████ 1400/1400 [03:28<00:00, 6.72it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:36<00:00, 6.33it/s]
	all	1866	108784	0.664	0.453	0.494 0.249
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
14/30	6.37G	1.943	1.176	1.135	156	640: 100% ██████████ 1400/1400 [03:27<00:00, 6.73it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:37<00:00, 6.27it/s]
	all	1866	108784	0.661	0.453	0.494 0.248
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
15/30	6.41G	1.916	1.159	1.123	11	640: 100% ██████████ 1400/1400 [03:27<00:00, 6.74it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:37<00:00, 6.28it/s]
	all	1866	108784	0.658	0.443	0.491 0.247
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
16/30	7.83G	1.934	1.153	1.131	87	640: 100% ██████████ 1400/1400 [03:29<00:00, 6.68it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:37<00:00, 6.29it/s]
	all	1866	108784	0.671	0.461	0.502 0.252
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
17/30	6.3G	1.903	1.125	1.12	189	640: 100% ██████████ 1400/1400 [03:30<00:00, 6.66it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:37<00:00, 6.16it/s]
	all	1866	108784	0.668	0.456	0.502 0.255
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
18/30	6.33G	1.898	1.133	1.124	54	640: 100% ██████████ 1400/1400 [03:29<00:00, 6.67it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:38<00:00, 6.14it/s]
	all	1866	108784	0.671	0.468	0.509 0.259
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
19/30	6.37G	1.915	1.122	1.115	97	640: 100% ██████████ 1400/1400 [03:28<00:00, 6.71it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:36<00:00, 6.49it/s]
	all	1866	108784	0.684	0.47	0.512 0.262
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
20/30	6.41G	1.885	1.108	1.112	12	640: 100% ██████████ 1400/1400 [03:36<00:00, 6.47it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:41<00:00, 5.69it/s]
	all	1866	108784	0.686	0.468	0.516 0.267

Closing dataloader mosaic

albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
21/30	6.44G	1.882	1.193	1.125	60	640: 100% 1400/1400 [03:39<00:00, 6.39it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:37<00:00, 6.19it/s]
	all	1866	108784	0.682	0.469	0.513 0.263
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
22/30	6.48G	1.86	1.181	1.121	1	640: 100% 1400/1400 [03:26<00:00, 6.77it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:35<00:00, 6.52it/s]
	all	1866	108784	0.67	0.464	0.51 0.261
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
23/30	6.52G	1.86	1.174	1.123	128	640: 100% 1400/1400 [03:32<00:00, 6.60it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.36it/s]
	all	1866	108784	0.681	0.475	0.518 0.265
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
24/30	6.55G	1.87	1.163	1.12	71	640: 100% 1400/1400 [03:27<00:00, 6.75it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:37<00:00, 6.22it/s]
	all	1866	108784	0.685	0.467	0.519 0.271
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
25/30	6.59G	1.849	1.15	1.111	68	640: 100% 1400/1400 [03:26<00:00, 6.77it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.42it/s]
	all	1866	108784	0.684	0.474	0.522 0.27
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
26/30	6.62G	1.831	1.136	1.104	1770	640: 100% 1400/1400 [03:27<00:00, 6.75it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.50it/s]
	all	1866	108784	0.684	0.472	0.519 0.268
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
27/30	6.66G	1.841	1.138	1.11	117	640: 100% 1400/1400 [03:27<00:00, 6.76it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:38<00:00, 6.11it/s]
	all	1866	108784	0.687	0.468	0.519 0.269
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
28/30	6.7G	1.813	1.127	1.105	19	640: 100% 1400/1400 [03:27<00:00, 6.76it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.45it/s]
	all	1866	108784	0.692	0.473	0.525 0.275
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
29/30	6.74G	1.815	1.108	1.104	26	640: 100% 1400/1400 [03:30<00:00, 6.64it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.38it/s]
	all	1866	108784	0.692	0.477	0.527 0.276
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
30/30	6.77G	1.807	1.108	1.099	26	640: 100% 1400/1400 [03:27<00:00, 6.75it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:36<00:00, 6.39it/s]
	all	1866	108784	0.695	0.478	0.53 0.279

30 epochs completed in 2.083 hours.
Optimizer stripped from /content/yolo_runs/building_detection_yolov8s/weights/last.pt, 22.5MB
Optimizer stripped from /content/yolo_runs/building_detection_yolov8s/weights/best.pt, 22.5MB

Validating /content/yolo_runs/building_detection_yolov8s/weights/best.pt...

Ultralytics 8.3.170 🚀 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 72 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	1866	108784	0.695	0.478	0.53	0.279

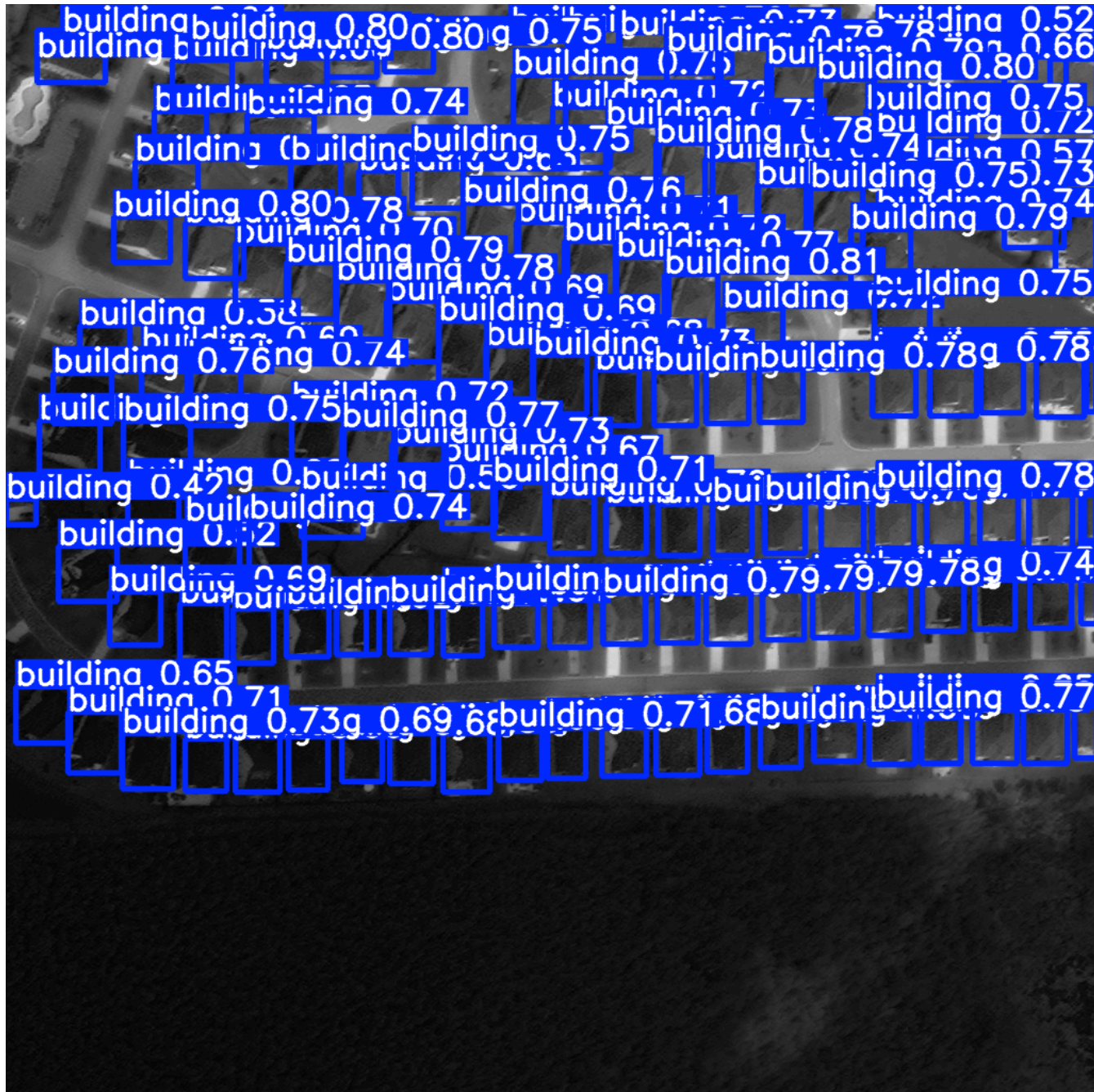
Speed: 0.4ms preprocess, 5.0ms inference, 0.0ms loss, 3.4ms postprocess per image

Results saved to /content/yolo_runs/building_detection_yolov8s

"""

🔗 ' Results from 30 epochs training\nUltralytics 8.3.170 🚀 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)\nengine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=4, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=/content/yolo_dataset/dataset.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=30, erasing=0.4, exist_ok=False, flipplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8s.pt, momentum=0.937, mosaic=1.0, multi_scale=False, name=building_detection_yolov8s, nbs=64, nms=False, opset=None, optimize=False, optimizer=aut

```
model = YOLO('/content/yolo_runs/building_detection_yolov8s/weights/best.pt')
results = model('/content/holdout_png/hurricane-florence_00000014_post_disaster.png')
results[0].show() # or .save() to export
```




```
import shutil

shutil.copytree(
    '/content/yolo_runs/building_detection_yolov8s',
    '/content/drive/MyDrive/YOLO_Backups/building_detection_yolov8s'
)

# Increase number of epochs to 50 as the model was still learning at 30 epochs
# Images are also kept to 1024x1024x3 (Original size) to better capture finer details
results = model.train(
    data='/content/yolo_dataset/dataset.yaml',
    epochs=50,
    imgsz=1024,
    batch=4,
    patience=5,
    name='building_detection_yolov8s_final',
    project='/content/yolo_runs_1024'
)
```

Results from 50 epochs training

```
# @title Results from 50 epochs training
"""
Ultralytics 8.3.177 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=4, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, conf=None, copy_paste=

from n  params module arguments
0 -1 1 928 ultralytics.nn.modules.conv.Conv [3, 32, 3, 2]
1 -1 1 18560 ultralytics.nn.modules.conv.Conv [32, 64, 3, 2]
2 -1 1 29056 ultralytics.nn.modules.block.C2f [64, 64, 1, True]
3 -1 1 73984 ultralytics.nn.modules.conv.Conv [64, 128, 3, 2]
4 -1 2 197632 ultralytics.nn.modules.block.C2f [128, 128, 2, True]
5 -1 1 295424 ultralytics.nn.modules.conv.Conv [128, 256, 3, 2]
6 -1 2 788480 ultralytics.nn.modules.block.C2f [256, 256, 2, True]
7 -1 1 1180672 ultralytics.nn.modules.conv.Conv [256, 512, 3, 2]
8 -1 1 1838080 ultralytics.nn.modules.block.C2f [512, 512, 1, True]
9 -1 1 656896 ultralytics.nn.modules.block.SPPF [512, 512, 5]
10 -1 1 0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
11 [-1, 6] 1 0 ultralytics.nn.modules.conv.Concat [1]
12 -1 1 591360 ultralytics.nn.modules.block.C2f [768, 256, 1]
13 -1 1 0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
14 [-1, 4] 1 0 ultralytics.nn.modules.conv.Concat [1]
15 -1 1 148224 ultralytics.nn.modules.block.C2f [384, 128, 1]
16 -1 1 147712 ultralytics.nn.modules.conv.Conv [128, 128, 3, 2]
17 [-1, 12] 1 0 ultralytics.nn.modules.conv.Concat [1]
18 -1 1 493056 ultralytics.nn.modules.block.C2f [384, 256, 1]
19 -1 1 590336 ultralytics.nn.modules.conv.Conv [256, 256, 3, 2]
20 [-1, 9] 1 0 ultralytics.nn.modules.conv.Concat [1]
21 -1 1 1969152 ultralytics.nn.modules.block.C2f [768, 512, 1]
22 [15, 18, 21] 1 2116435 ultralytics.nn.modules.head.Detect [1, [128, 256, 512]]

Model summary: 129 layers, 11,135,987 parameters, 11,135,971 gradients, 28.6 GFLOPs

Transferred 355/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
```

AMP: running Automatic Mixed Precision (AMP) checks...

AMP: checks passed 

train: Fast image access  (ping: 0.0±0.0 ms, read: 2451.3±521.1 MB/s, size: 544.5 KB)

train: Scanning /content/yolo_dataset/labels/train.cache... 5598 images, 1032 backgrounds, 0 corrupt: 100% 5598/5598 [00:00<?, ?it/s]train: /content/yolo_dataset/images/train/palu-tsunami_

train: /content/yolo_dataset/images/train/palu-tsunami_00000106_pre_disaster.png: 1 duplicate labels removed

albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid

val: Fast image access  (ping: 0.0±0.0 ms, read: 1255.8±1379.3 MB/s, size: 558.2 KB)

val: Scanning /content/yolo_dataset/labels/val.cache... 1866 images, 340 backgrounds, 0 corrupt: 100% 1866/1866 [00:00<?, ?it/s]

Plotting labels to /content/yolo_runs_1024/building_detection_yolov8s_final/labels.jpg...

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...

optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)

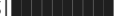
Image sizes 1024 train, 1024 val

Using 2 dataloader workers

Logging results to /content/yolo_runs_1024/building_detection_yolov8s_final

Starting training for 50 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/50	10.6G	1.934	1.488	1.313	234	1024: 100%  1400/1400 [05:49<00:00, 4.00it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [01:01<00:00, 3.78it/s]
	all	1866	108784	0.573	0.39	0.407 0.204
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/50	10.9G	1.947	1.461	1.329	352	1024: 100%  1400/1400 [05:42<00:00, 4.09it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:54<00:00, 4.27it/s]
	all	1866	108784	0.581	0.424	0.435 0.218
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/50	14.1G	1.936	1.425	1.312	354	1024: 100%  1400/1400 [05:41<00:00, 4.10it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:56<00:00, 4.17it/s]
	all	1866	108784	0.616	0.435	0.461 0.235
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
4/50	9.39G	1.911	1.385	1.304	88	1024: 100%  1400/1400 [05:44<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:59<00:00, 3.93it/s]
	all	1866	108784	0.634	0.46	0.487 0.251
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/50	9.32G	1.882	1.362	1.292	69	1024: 100%  1400/1400 [05:46<00:00, 4.04it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:54<00:00, 4.26it/s]
	all	1866	108784	0.656	0.467	0.503 0.26
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
6/50	12.5G	1.86	1.3	1.282	135	1024: 100%  1400/1400 [05:46<00:00, 4.04it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:58<00:00, 4.03it/s]
	all	1866	108784	0.662	0.474	0.507 0.268
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
7/50	7.32G	1.838	1.273	1.264	335	1024: 100%  1400/1400 [05:48<00:00, 4.02it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:56<00:00, 4.14it/s]
	all	1866	108784	0.642	0.463	0.496 0.261
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
8/50	12.2G	1.836	1.266	1.262	140	1024: 100%  1400/1400 [05:45<00:00, 4.05it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:55<00:00, 4.18it/s]
	all	1866	108784	0.66	0.461	0.499 0.267

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
9/50	12.2G	1.812	1.235	1.249	33	1024: 100%  1400/1400 [05:45<00:00, 4.06it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:59<00:00, 3.95it/s]
	all	1866	108784	0.672	0.478	0.517 0.281
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
10/50	10.2G	1.815	1.224	1.247	422	1024: 100%  1400/1400 [05:43<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:56<00:00, 4.11it/s]
	all	1866	108784	0.65	0.477	0.505 0.271
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
11/50	14.4G	1.801	1.213	1.244	160	1024: 100%  1400/1400 [05:45<00:00, 4.05it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:59<00:00, 3.94it/s]
	all	1866	108784	0.676	0.485	0.526 0.285
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
12/50	10.4G	1.786	1.203	1.235	375	1024: 100%  1400/1400 [05:43<00:00, 4.08it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:56<00:00, 4.15it/s]
	all	1866	108784	0.694	0.488	0.535 0.29
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
13/50	11.7G	1.776	1.197	1.225	30	1024: 100%  1400/1400 [05:41<00:00, 4.10it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:54<00:00, 4.31it/s]
	all	1866	108784	0.696	0.495	0.539 0.292
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
14/50	5.68G	1.76	1.172	1.219	156	1024: 100%  1400/1400 [05:44<00:00, 4.06it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:57<00:00, 4.09it/s]
	all	1866	108784	0.696	0.499	0.542 0.299
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
15/50	12.7G	1.743	1.153	1.213	11	1024: 100%  1400/1400 [05:46<00:00, 4.04it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:55<00:00, 4.21it/s]
	all	1866	108784	0.706	0.493	0.545 0.295
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
16/50	10.4G	1.758	1.157	1.221	87	1024: 100%  1400/1400 [05:41<00:00, 4.10it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:55<00:00, 4.23it/s]
	all	1866	108784	0.688	0.504	0.543 0.298
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
17/50	11.8G	1.735	1.128	1.21	190	1024: 100%  1400/1400 [05:43<00:00, 4.08it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:57<00:00, 4.05it/s]
	all	1866	108784	0.699	0.499	0.545 0.3
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
18/50	11.2G	1.724	1.137	1.213	54	1024: 100%  1400/1400 [05:42<00:00, 4.09it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:55<00:00, 4.23it/s]
	all	1866	108784	0.69	0.5	0.544 0.297
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
19/50	11.4G	1.745	1.127	1.205	97	1024: 100%  1400/1400 [05:45<00:00, 4.06it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100%  234/234 [00:56<00:00, 4.13it/s]
	all	1866	108784	0.69	0.505	0.547 0.302

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
20/50	12.6G	1.725	1.12	1.202	12	1024: 100% ██████████ 1400/1400 [06:06<00:00, 3.82it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:56<00:00, 4.16it/s]
	all	1866	108784	0.703	0.502	0.551 0.305
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
21/50	11G	1.712	1.113	1.196	34	1024: 100% ██████████ 1400/1400 [05:42<00:00, 4.08it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:56<00:00, 4.15it/s]
	all	1866	108784	0.684	0.5	0.545 0.304
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
22/50	11.3G	1.701	1.098	1.19	175	1024: 100% ██████████ 1400/1400 [05:44<00:00, 4.06it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:55<00:00, 4.21it/s]
	all	1866	108784	0.705	0.51	0.554 0.309
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
23/50	10G	1.708	1.101	1.199	288	1024: 100% ██████████ 1400/1400 [05:48<00:00, 4.02it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:54<00:00, 4.26it/s]
	all	1866	108784	0.701	0.51	0.554 0.308
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
24/50	12.2G	1.703	1.1	1.192	210	1024: 100% ██████████ 1400/1400 [05:43<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:55<00:00, 4.21it/s]
	all	1866	108784	0.713	0.507	0.559 0.311
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
25/50	11.1G	1.698	1.089	1.193	25	1024: 100% ██████████ 1400/1400 [05:45<00:00, 4.05it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:55<00:00, 4.23it/s]
	all	1866	108784	0.717	0.513	0.563 0.315
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
26/50	11.6G	1.699	1.088	1.188	16	1024: 100% ██████████ 1400/1400 [05:45<00:00, 4.05it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:56<00:00, 4.17it/s]
	all	1866	108784	0.719	0.515	0.567 0.32
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
27/50	13.2G	1.682	1.071	1.177	58	1024: 100% ██████████ 1400/1400 [05:44<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:54<00:00, 4.26it/s]
	all	1866	108784	0.714	0.518	0.569 0.321
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
28/50	11.6G	1.683	1.076	1.184	51	1024: 100% ██████████ 1400/1400 [05:42<00:00, 4.09it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:57<00:00, 4.04it/s]
	all	1866	108784	0.72	0.517	0.571 0.323
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
29/50	9.08G	1.685	1.076	1.182	168	1024: 100% ██████████ 1400/1400 [05:41<00:00, 4.10it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:55<00:00, 4.23it/s]
	all	1866	108784	0.713	0.519	0.57 0.321
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
30/50	13.1G	1.668	1.062	1.172	64	1024: 100% ██████████ 1400/1400 [05:43<00:00, 4.08it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ██████████ 234/234 [00:55<00:00, 4.24it/s]
	all	1866	108784	0.718	0.517	0.569 0.322
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size

31/50	12.4G	1.667	1.059	1.173	211	1024: 100% 1400/1400 [05:41<00:00, 4.10it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.25it/s]
	all	1866	108784	0.71	0.519	0.569 0.322
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
32/50	8.77G	1.675	1.062	1.177	44	1024: 100% 1400/1400 [05:50<00:00, 3.99it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:56<00:00, 4.14it/s]
	all	1866	108784	0.716	0.521	0.575 0.327
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
33/50	11G	1.652	1.045	1.171	382	1024: 100% 1400/1400 [05:44<00:00, 4.06it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.20it/s]
	all	1866	108784	0.711	0.525	0.574 0.325
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
34/50	9.76G	1.649	1.04	1.166	163	1024: 100% 1400/1400 [05:43<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:56<00:00, 4.18it/s]
	all	1866	108784	0.724	0.525	0.576 0.326
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
35/50	9.43G	1.637	1.029	1.16	43	1024: 100% 1400/1400 [05:44<00:00, 4.07it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:57<00:00, 4.08it/s]
	all	1866	108784	0.716	0.531	0.579 0.329
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
36/50	12.8G	1.64	1.031	1.162	47	1024: 100% 1400/1400 [05:49<00:00, 4.01it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:56<00:00, 4.11it/s]
	all	1866	108784	0.72	0.525	0.578 0.331
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
37/50	8.93G	1.636	1.024	1.16	169	1024: 100% 1400/1400 [05:51<00:00, 3.99it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:53<00:00, 4.34it/s]
	all	1866	108784	0.72	0.526	0.578 0.33
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
38/50	9.8G	1.634	1.023	1.156	165	1024: 100% 1400/1400 [05:47<00:00, 4.03it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:54<00:00, 4.33it/s]
	all	1866	108784	0.717	0.528	0.579 0.331
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
39/50	14.1G	1.631	1.018	1.158	132	1024: 100% 1400/1400 [05:43<00:00, 4.08it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:53<00:00, 4.37it/s]
	all	1866	108784	0.727	0.527	0.582 0.333
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
40/50	11.5G	1.616	1.017	1.152	54	1024: 100% 1400/1400 [05:45<00:00, 4.05it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.20it/s]
	all	1866	108784	0.723	0.53	0.581 0.332

Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
41/50	13.3G	1.627	1.076	1.166	107	1024: 100% 1400/1400 [05:31<00:00, 4.23it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.20it/s]
	all	1866	108784	0.725	0.524	0.578 0.33

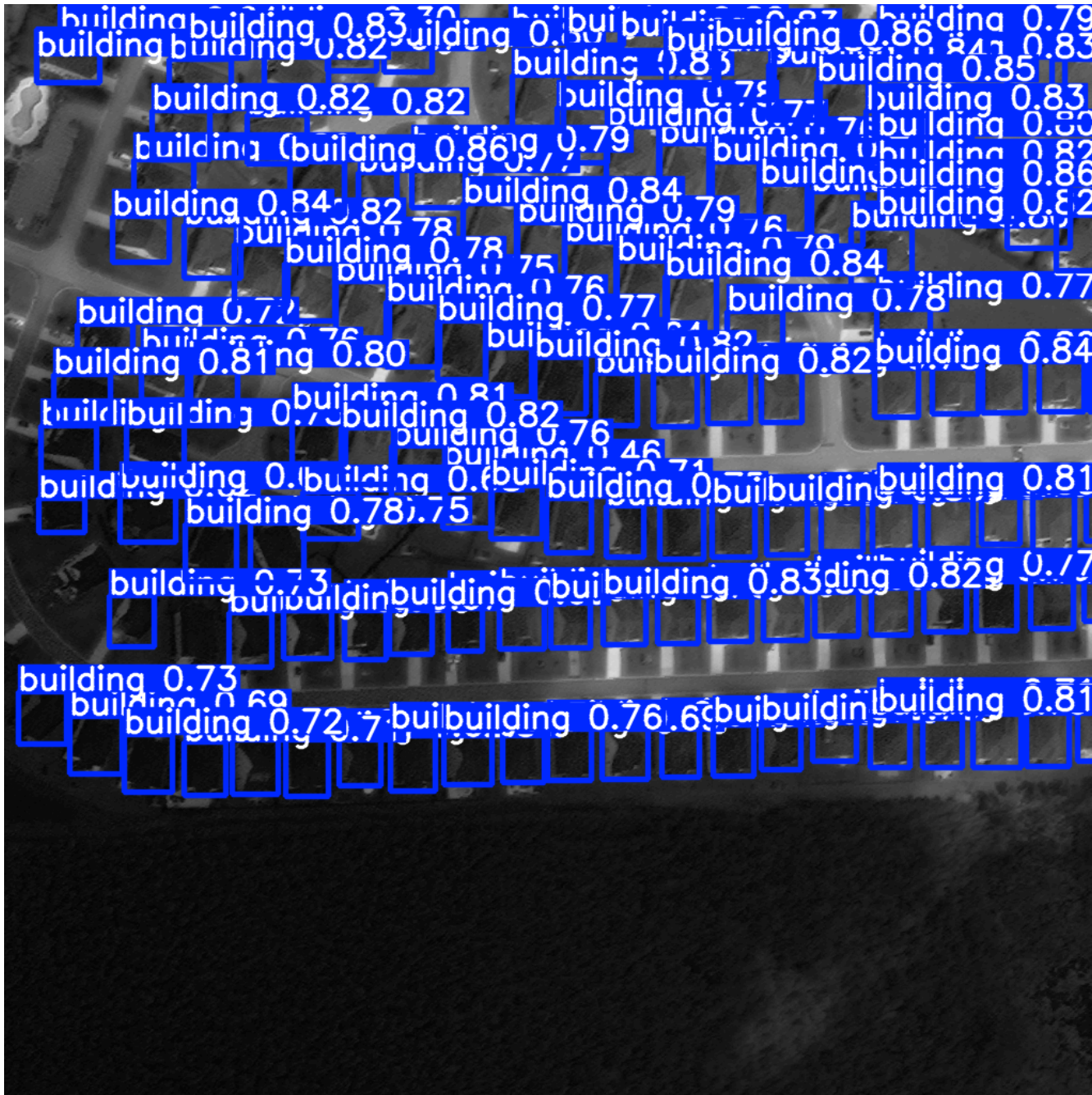
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
42/50	10.5G	1.611	1.061	1.158	26	1024: 100% 1400/1400 [05:24<00:00, 4.32it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.23it/s]
	all	1866	108784	0.725	0.527	0.58 0.332
43/50	11.6G	1.615	1.05	1.159	206	1024: 100% 1400/1400 [05:36<00:00, 4.16it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:55<00:00, 4.22it/s]
	all	1866	108784	0.73	0.528	0.584 0.335
44/50	10.1G	1.61	1.047	1.159	87	1024: 100% 1400/1400 [05:20<00:00, 4.37it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:54<00:00, 4.28it/s]
	all	1866	108784	0.724	0.529	0.581 0.334
45/50	8.54G	1.587	1.027	1.152	10	1024: 100% 1400/1400 [05:21<00:00, 4.35it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:53<00:00, 4.38it/s]
	all	1866	108784	0.725	0.529	0.582 0.334
46/50	11.9G	1.586	1.027	1.154	115	1024: 100% 1400/1400 [05:21<00:00, 4.35it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:53<00:00, 4.38it/s]
	all	1866	108784	0.724	0.53	0.582 0.335
47/50	7.38G	1.592	1.032	1.152	445	1024: 100% 1400/1400 [05:18<00:00, 4.39it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:54<00:00, 4.31it/s]
	all	1866	108784	0.726	0.53	0.582 0.334
48/50	10.6G	1.586	1.015	1.147	1	1024: 100% 1400/1400 [05:20<00:00, 4.37it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:54<00:00, 4.33it/s]
	all	1866	108784	0.728	0.53	0.583 0.336
49/50	11.2G	1.583	1.026	1.142	64	1024: 100% 1400/1400 [05:21<00:00, 4.36it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:53<00:00, 4.39it/s]
	all	1866	108784	0.732	0.529	0.585 0.336
50/50	12.5G	1.587	1.014	1.15	981	1024: 100% 1400/1400 [05:21<00:00, 4.35it/s]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 234/234 [00:54<00:00, 4.28it/s]
	all	1866	108784	0.729	0.531	0.585 0.336

50 epochs completed in 5.550 hours.
Optimizer stripped from /content/yolo_runs_1024/building_detection_yolov8s_final/weights/last.pt, 22.6MB
Optimizer stripped from /content/yolo_runs_1024/building_detection_yolov8s_final/weights/best.pt, 22.6MB

Validating /content/yolo_runs_1024/building_detection_yolov8s_final/weights/best.pt...
Ultralytics 8.3.177 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
Model summary (fused): 72 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% | 234/234 [00:50<00:00, 4.62it/s]
all 1866 108784 0.729 0.531 0.585 0.336
Speed: 0.7ms preprocess, 10.8ms inference, 0.0ms loss, 5.2ms postprocess per image

```
Results saved to /content/yolo_runs_1024/building_detection_yolov8s_final
"""
```

```
model = YOLO('/content/yolo_runs_1024/building_detection_yolov8s_final/weights/best.pt')
results = model('/content/holdout_png/hurricane-florence_00000014_post_disaster.png')
results[0].show() # or .save() to export
```




```
import shutil

shutil.copytree(
    '/content/yolo_runs_1024/building_detection_yolov8s_final',
    '/content/drive/MyDrive/YOLO_Backups_final/building_detection_yolov8s_final'
)
```

Start coding or [generate](#) with AI.

✓ Test

```
!pip install ultralytics
import zipfile
from ultralytics import YOLO
import os
import glob
from PIL import Image
import cv2
import numpy as np
from tqdm import tqdm
import json
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from ultralytics import YOLO
```

 [Show hidden output](#)

```
# Extract Test
test_base = '/content/drive/MyDrive/Building Damage Assessment/Test'
test_images_zip = os.path.join(test_base, 'images.zip')
test_labels_zip = os.path.join(test_base, 'labels.zip')
```

```
if os.path.exists(test_images_zip):
    with zipfile.ZipFile(test_images_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/test_images')
    print('Test images extracted')
```

```
if os.path.exists(test_labels_zip):
    with zipfile.ZipFile(test_labels_zip, 'r') as zip_ref:
        zip_ref.extractall('/content/test_labels')
    print('Test labels extracted')
```

 Test images extracted
Test labels extracted

```
def analyze_tiff_file(file_path):
    """Analyze a TIFF file to understand its properties"""
    print(f"\nAnalyzing: {os.path.basename(file_path)}")
```

```
# Check file size
file_size = os.path.getsize(file_path)
```

```

print(f"File size: {file_size} bytes")

# Try to read first few bytes
try:
    with open(file_path, 'rb') as f:
        header = f.read(16)
        print(f"Header bytes: {header[:8].hex()}")

        # TIFF files should start with either:
        # II* (Intel/little-endian): 49 49 2A 00
        # MM* (Motorola/big-endian): 4D 4D 00 2A
        if header[:4] in [b'II*\x00', b'MM*\x00*']:
            print("Valid TIFF header detected")
        else:
            print("Invalid TIFF header - file may be corrupted")
except Exception as e:
    print(f"Error reading file header: {e}")

def convert_with_opencv(input_path, output_path):
    """Try converting using OpenCV"""
    try:
        # Read with OpenCV
        img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)
        if img is None:
            return False, "OpenCV couldn't read the image"

        # Convert BGR to RGB if needed
        if len(img.shape) == 3 and img.shape[2] == 3:
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Save as PNG using OpenCV
        cv2.imwrite(output_path, img)
        return True, "Success with OpenCV"
    except Exception as e:
        return False, f"OpenCV error: {e}"

def convert_with_pillow_force(input_path, output_path):
    """Try converting with PIL using different modes"""
    methods = [
        lambda: Image.open(input_path),
        lambda: Image.open(input_path).convert('RGB'),
        lambda: Image.open(input_path).convert('RGBA'),
    ]

    for i, method in enumerate(methods):
        try:
            img = method()
            img.save(output_path, 'PNG')
            return True, f"Success with PIL method {i+1}"
        except Exception as e:
            continue

    return False, "All PIL methods failed"

def repair_and_convert_tiff(input_folder, output_folder, sample_file=None):
    """

```

```

Robust TIFF to PNG conversion with multiple fallback methods
"""
# Create output folder
os.makedirs(output_folder, exist_ok=True)

# Find all TIFF files
tiff_patterns = ['*.tif', '*.tiff', '*.TIF', '*.TIFF']
tiff_files = []

for pattern in tiff_patterns:
    tiff_files.extend(glob.glob(os.path.join(input_folder, '**', pattern), recursive=True))

print(f"Found {len(tiff_files)} TIFF files")

# If sample_file is specified, analyze just that one
if sample_file:
    sample_path = None
    for tiff_file in tiff_files:
        if sample_file in os.path.basename(tiff_file):
            sample_path = tiff_file
            break

    if sample_path:
        analyze_tiff_file(sample_path)
        return
    else:
        print(f"Sample file '{sample_file}' not found")
        return

for i, tiff_file in enumerate(tiff_files[:3]):
    analyze_tiff_file(tiff_file)
    if i < 2: # Don't print separator after last file
        print("-" * 30)

# Since files have valid headers, proceed with conversion
proceed = True

# Attempt conversion with multiple methods
converted_count = 0
failed_files = []

for tiff_file in tqdm(tiff_files, desc="Converting files"):
    # Get relative path and create output path
    rel_path = os.path.relpath(tiff_file, input_folder)
    png_filename = os.path.splitext(rel_path)[0] + '.png'
    output_path = os.path.join(output_folder, png_filename)

    # Create output directory
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    # Try different conversion methods
    methods = [
        ("OpenCV", lambda: convert_with_opencv(tiff_file, output_path)),
        ("PIL Force", lambda: convert_with_pillow_force(tiff_file, output_path)),
    ]

```

```

        converted = False
        for method_name, method_func in methods:
            try:
                success, message = method_func()
                if success:
                    converted_count += 1
                    converted = True
                    break
            except Exception as e:
                continue

        if not converted:
            failed_files.append(tiff_file)

    print(f"\nConversion Results:")
    print(f"Successfully converted: {converted_count}")
    print(f"Failed: {len(failed_files)}")

    if failed_files:
        print(f"\nFirst 5 failed files:")
        for failed_file in failed_files[:5]:
            print(f" {os.path.basename(failed_file)}")

repair_and_convert_tiff(
    input_folder='/content/test_images/images',
    output_folder='/content/test_png'
)

🔗 Found 1866 TIFF files

Analyzing: social-fire_00000431_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----

Analyzing: hurricane-florence_00000141_post_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
-----

Analyzing: hurricane-matthew_00000122_pre_disaster.tif
File size: 6304308 bytes
Header bytes: 49492a00aa186000
Valid TIFF header detected
Converting files: 100%[██████████] 1866/1866 [01:36<00:00, 19.37it/s]\nConversion Results:
Successfully converted: 1866
Failed: 0

def convert_coco_to_yolo_bbox(bbox, img_width, img_height):
    """
    Convert COCO bbox format to YOLO format
    COCO format: [x, y, width, height] (absolute coordinates)
    YOLO format: [center_x, center_y, width, height] (normalized 0-1)
    """

```

```

    x, y, width, height = bbox

    # Convert to center coordinates
    center_x = x + width / 2
    center_y = y + height / 2

    # Normalize by image dimensions
    center_x /= img_width
    center_y /= img_height
    width /= img_width
    height /= img_height

    return [center_x, center_y, width, height]

def convert_coco_to_yolo(coco_annotation_file, yolo_labels_dir, images_info=None):
    """
    Convert COCO format annotations to YOLO format

    Args:
        coco_annotation_file: Path to COCO JSON file
        yolo_labels_dir: Directory to save YOLO format .txt files
        images_info: Optional dict mapping image_id to image info
    """
    # Create output directory
    os.makedirs(yolo_labels_dir, exist_ok=True)

    # Load COCO annotations
    with open(coco_annotation_file, 'r') as f:
        coco_data = json.load(f)

    # Create mapping of image_id to image info
    if images_info is None:
        images_info = {img['id']: img for img in coco_data['images']}

    # Create mapping of image_id to filename (without extension)
    image_id_to_filename = {}
    for img in coco_data['images']:
        # Remove .tif extension and replace with .png since we converted images
        base_name = os.path.splitext(img['file_name'])[0]
        image_id_to_filename[img['id']] = base_name

    # Group annotations by image_id
    annotations_by_image = {}
    for ann in coco_data['annotations']:
        image_id = ann['image_id']
        if image_id not in annotations_by_image:
            annotations_by_image[image_id] = []
        annotations_by_image[image_id].append(ann)

    print(f"Converting {len(coco_data['annotations'])} annotations for {len(coco_data['images'])} images...")

    converted_count = 0

    # Convert each image's annotations
    for image_id, img_info in tqdm(images_info.items(), desc="Converting to YOLO format"):
        filename_base = image_id_to_filename.get(image_id, f"image_{image_id}")

```

```

yolo_file_path = os.path.join(yolo_labels_dir, f"{filename_base}.txt")

# Get annotations for this image
image_annotations = annotations_by_image.get(image_id, [])

# Write YOLO format annotations
yolo_lines = []
for ann in image_annotations:
    # Convert bbox to YOLO format
    yolo_bbox = convert_coco_to_yolo_bbox(
        ann['bbox'],
        img_info['width'],
        img_info['height']
    )

    # YOLO format: class_id center_x center_y width height
    # Since you only have 'building' class, class_id = 0
    class_id = 0 # Buildings are class 0
    yolo_line = f"{class_id} {yolo_bbox[0]:.6f} {yolo_bbox[1]:.6f} {yolo_bbox[2]:.6f} {yolo_bbox[3]:.6f}"
    yolo_lines.append(yolo_line)

# Write to file (even if empty - YOLO expects a file for each image)
with open(yolo_file_path, 'w') as f:
    f.write('\n'.join(yolo_lines))
    if yolo_lines: # Add final newline if there are lines
        f.write('\n')

converted_count += 1

print(f"Converted {converted_count} label files to YOLO format")
print(f"Labels saved to: {yolo_labels_dir}")

return converted_count

def convert_annotations_to_yolo(test_annotation_file, yolo_labels_dir):
    """Convert test annotations (custom format) to YOLO format"""
    import json
    import os
    from tqdm import tqdm

    # Create output directory
    os.makedirs(yolo_labels_dir, exist_ok=True)

    # Load test annotations
    with open(test_annotation_file, 'r') as f:
        test_data = json.load(f)

    converted_count = 0

    print(f"Processing {len(test_data)} images")

    for image_filename, annotations in tqdm(test_data.items(), desc="Converting annotations"):
        # Get base filename without extension for YOLO label file
        base_filename = os.path.splitext(image_filename)[0]
        # Convert .tif to .png in the base filename if needed
        base_filename = base_filename.replace('.tif', '.png')

```

```

yolo_filename = f"{base_filename}.txt"
yolo_filepath = os.path.join(yolo_labels_dir, yolo_filename)

# Convert annotations to YOLO format
yolo_lines = []

for annotation in annotations:
    bbox = annotation['bbox']
    label = annotation.get('label', 0) # Default to class 0 if no label

    # Assume bbox is in format [x_min, y_min, width, height] (COCO format)
    # Need to convert to YOLO format [class, x_center_norm, y_center_norm, width_norm, height_norm]

    # For now, assume image dimensions - you might need to get actual dimensions
    # You can modify this if you have access to image dimensions
    img_width = 1024 # Default assumption - adjust if you know the actual dimensions
    img_height = 1024

    x_min, y_min, width, height = bbox

    # Convert to YOLO format (normalized coordinates)
    x_center = (x_min + width / 2) / img_width
    y_center = (y_min + height / 2) / img_height
    width_norm = width / img_width
    height_norm = height / img_height

    # YOLO format: class x_center y_center width height (all normalized)
    yolo_line = f"{label} {x_center:.6f} {y_center:.6f} {width_norm:.6f} {height_norm:.6f}"
    yolo_lines.append(yolo_line)

# Write YOLO annotation file
if yolo_lines: # Only create file if there are annotations
    with open(yolo_filepath, 'w') as f:
        f.write('\n'.join(yolo_lines))
    converted_count += 1

print(f"Successfully converted {converted_count} annotation files")
return converted_count

def setup_yolo_test_dataset_with_labels():
    """Setup of YOLO test dataset with converted labels"""

    print("="*60)
    print("SETTING UP YOLO TEST DATASET WITH LABELS")
    print("="*60)

    # Paths
    PROCESSED_DATA = '/content/drive/MyDrive/Building Damage Assessment/Annotations'
    YOLO_DATASET_ROOT = '/content/yolo_dataset'
    TEST_IMAGES_SOURCE = '/content/test_png' # Adjust this path as needed

    # Create YOLO test directory structure
    os.makedirs(YOLO_DATASET_ROOT, exist_ok=True)
    os.makedirs(f'{YOLO_DATASET_ROOT}/images/test', exist_ok=True)
    os.makedirs(f'{YOLO_DATASET_ROOT}/labels/test', exist_ok=True)

```

```

print(f"\n{'='*30}")
print("PROCESSING TEST SPLIT")
print(f"{'='*30}")

# Convert test annotations from custom format to YOLO format
test_coco_file = f'{PROCESSED_DATA}/test_annotations.json' # Adjust path as needed
test_labels_dir = f'{YOLO_DATASET_ROOT}/labels/test'

if os.path.exists(test_coco_file):
    print("Converting test annotations to YOLO format")
    converted_count = convert_test_annotations_to_yolo(test_coco_file, test_labels_dir)
    print(f"Converted {converted_count} annotation files")
else:
    print(f"WARNING: Test annotation file not found: {test_coco_file}")
    print("Please update the path to your test annotations file")

# Organize test images
if os.path.exists(TEST_IMAGES_SOURCE):
    import glob
    import shutil
    from tqdm import tqdm

    png_files = glob.glob(os.path.join(TEST_IMAGES_SOURCE, '**', '*.png'), recursive=True)
    print(f"Found {len(png_files)} PNG test images to organize")

    if png_files:
        linked_count = 0
        images_dest = f'{YOLO_DATASET_ROOT}/images/test'

        # Show progress for first 10 files
        for png_file in tqdm(png_files[:10], desc="Organizing test images"):
            filename = os.path.basename(png_file)
            dest_path = os.path.join(images_dest, filename)

            try:
                if not os.path.exists(dest_path):
                    os.symlink(png_file, dest_path)
                    linked_count += 1
            except:
                try:
                    shutil.copy2(png_file, dest_path)
                    linked_count += 1
                except:
                    print(f"Failed to copy {filename}")

        # Process remaining files without progress bar for speed
        if len(png_files) > 10:
            for png_file in png_files[10:]:
                filename = os.path.basename(png_file)
                dest_path = os.path.join(images_dest, filename)

                try:
                    if not os.path.exists(dest_path):
                        os.symlink(png_file, dest_path)
                        linked_count += 1

```



```

        except:
            try:
                shutil.copy2(png_file, dest_path)
                linked_count += 1
            except:
                pass

        print(f"Organized {linked_count} test images")
    else:
        print("No PNG files found in test source directory")
else:
    print(f"WARNING: Test images source not found: {TEST_IMAGES_SOURCE}")

print("CREATING TEST DATASET.YAML")

dataset_config = {
    'path': YOLO_DATASET_ROOT,
    'test': 'images/test',
    'nc': 1, # number of classes
    'names': ['building'] # class names
}

import yaml
dataset_yaml_path = os.path.join(YOLO_DATASET_ROOT, 'dataset.yaml')
with open(dataset_yaml_path, 'w') as f:
    yaml.dump(dataset_config, f, default_flow_style=False)

print(f"Created dataset.yaml at: {dataset_yaml_path}")

test_images_dir = f'{YOLO_DATASET_ROOT}/images/test'
test_labels_dir = f'{YOLO_DATASET_ROOT}/labels/test'

if os.path.exists(test_images_dir) and os.path.exists(test_labels_dir):
    import glob
    images_count = len(glob.glob(os.path.join(test_images_dir, '*.png')))
    labels_count = len(glob.glob(os.path.join(test_labels_dir, '*.txt')))

setup_yolo_test_dataset_with_labels()

=====
SETTING UP YOLO TEST DATASET WITH LABELS
=====

=====
PROCESSING TEST SPLIT
=====
Converting test annotations to YOLO format
Processing 1504 test images...
Converting annotations: 100%|██████████| 1504/1504 [00:00<00:00, 3797.65it/s]
Successfully converted 1504 annotation files
Converted 1504 annotation files
Found 1866 PNG test images to organize
Organizing test images: 100%|██████████| 10/10 [00:00<00:00, 13490.85it/s]Organized 1866 test images
CREATING TEST DATASET.YAML
Created dataset.yaml at: /content/yolo_dataset/dataset.yaml

```

```

def run_test(model_path, test_images_dir, output_dir):
    """Run YOLO inference on test images"""

    # Load your trained model
    model = YOLO(model_path)
    # Load images
    test_images = glob.glob(f"{test_images_dir}/*.png")
    print(f"Found {len(test_images)} test images")
    # Output directory
    Path(output_dir).mkdir(parents=True, exist_ok=True)

    results_data = []

    for img_path in tqdm(test_images, desc="Running inference"):
        results = model(img_path)

        # Save prediction image
        img_name = Path(img_path).stem
        output_path = f"{output_dir}/{img_name}_pred.jpg"
        results[0].save(output_path)

        # Store detection data
        boxes = results[0].boxes
        if boxes is not None:
            detections = {
                'image': Path(img_path).name,
                'num_detections': len(boxes),
                'confidences': boxes.conf.cpu().numpy().tolist(),
                'boxes': boxes.xyxy.cpu().numpy().tolist()
            }
        else:
            detections = {
                'image': Path(img_path).name,
                'num_detections': 0,
                'confidences': [],
                'boxes': []
            }

        results_data.append(detections)

    # Save results to JSON
    with open(f"{output_dir}/test_results.json", 'w') as f:
        json.dump(results_data, f, indent=2)

    print(f"Results saved to {output_dir}")
    return results_data

def run_yolo_validation(model_path, test_dataset_yaml):
    """Run YOLO's built-in validation - gives you mAP, precision, recall automatically"""
    try:
        model = YOLO(model_path)

        # Run validation on test set
        metrics = model.val(data=test_dataset_yaml, split='test')

        # Extract metrics

```

```

precision = metrics.box.p[0] if len(metrics.box.p) > 0 else 0
recall = metrics.box.r[0] if len(metrics.box.r) > 0 else 0

# Calculate F1 score
f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else

return {
    'mAP50': float(metrics.box.map50),
    'mAP50_95': float(metrics.box.map),
    'precision': float(precision),
    'recall': float(recall),
    'f1_score': float(f1_score),
    'success': True
}
except Exception as e:
    return {
        'error': str(e),
        'success': False
    }

# Calculate additional metrics
def calculate_detection_rate(results_data):
    """How well does the model find buildings in images?"""

    total_images = len(results_data)
    images_with_detections = sum(1 for r in results_data if r['num_detections'] > 0)

    return {
        'detection_rate': images_with_detections / total_images if total_images > 0 else 0,
        'images_detected': images_with_detections,
        'total_images': total_images
    }

def calculate_confidence_quality(results_data, high_conf_threshold=0.7):
    """How confident is the model in its predictions?"""

    all_confidences = []
    for r in results_data:
        all_confidences.extend(r['confidences'])

    if not all_confidences:
        return {
            'avg_confidence': 0,
            'high_confidence_rate': 0,
            'total_detections': 0,
            'confidence_std': 0
        }

    high_conf_count = sum(1 for c in all_confidences if c >= high_conf_threshold)

    return {
        'avg_confidence': np.mean(all_confidences),
        'high_confidence_rate': high_conf_count / len(all_confidences),
        'total_detections': len(all_confidences),
        'confidence_std': np.std(all_confidences)
    }

```

```

def calculate_building_count_stats(results_data):
    """Building count statistics"""

    detection_counts = [r['num_detections'] for r in results_data]

    return {
        'avg_buildings_per_image': np.mean(detection_counts),
        'max_buildings_per_image': np.max(detection_counts),
        'std_buildings_per_image': np.std(detection_counts),
        'images_with_no_buildings': sum(1 for c in detection_counts if c == 0)
    }

# Print comprehensive results
def print_comprehensive_metrics(model_path, test_dataset_yaml, results_data):
    """Calculate and print all essential metrics including F1 score"""

    print("\n" + "="*70)
    print("COMPREHENSIVE TEST EVALUATION RESULTS")
    print("="*70)

    # 1. YOLO Validation Metrics (mAP, Precision, Recall, F1)
    print("\033[34mDETECTION ACCURACY METRICS")
    print("-"*40)
    yolo_metrics = run_yolo_validation(model_path, test_dataset_yaml)

    if yolo_metrics['success']:
        print(f"mAP@0.5:      {yolo_metrics['mAP50']:.3f}")
        print(f"mAP@0.5:0.95: {yolo_metrics['mAP50_95']:.3f}")
        print(f"Precision:     {yolo_metrics['precision']:.3f}")
        print(f"Recall:        {yolo_metrics['recall']:.3f}")
        print(f"F1 Score:      {yolo_metrics['f1_score']:.3f}")

        # Performance interpretation
        if yolo_metrics['f1_score'] > 0.7:
            print("Excellent F1 score")
        elif yolo_metrics['f1_score'] > 0.5:
            print("Good F1 score")
        else:
            print("F1 score needs improvement")

    else:
        print(f"Could not calculate YOLO metrics: {yolo_metrics['error']}")

# MAIN EXECUTION FUNCTION
def complete_test_evaluation(model_path, test_images_dir, test_dataset_yaml, output_dir):
    results_data = run_test(model_path, test_images_dir, output_dir)
    all_metrics = print_comprehensive_metrics(model_path, test_dataset_yaml, results_data)
    create_test_visualizations(results_data, output_dir)

# Run complete evaluation
model_path = '/content/drive/MyDrive/YOLO_Backups_final/building_detection_yolov8s_final/weights/best.pt'
test_images_dir = '/content/yolo_dataset/images/test'

```